A PROJECT REPORT ON

"CREDO SCORE: A LOAN DEFAULT RISK PREDICTOR"

Submitted in partial fulfilment of the requirements for the

award of the degree

OF

BACHELOR OF COMPUTER APPLICATIONS (BCA)

To

GURU GOBIND SINGH INDRAPRASTHA UNIVERSITY, DELHI

SUBMITTED BY:

GURSIMRAN KAUR

02490202024

UNDER THE SUPERVISION OF

MS. NISHA CHOUDHARY

(ASSISTANT PROFESSOR)



SRI GURU TEGH BAHADUR INSTITUTE OF MANAGEMENT AND INFORMATION TECHNOLOGY

(24, GT Karnal Rd, Gurdwara Nanak Piao Campus, Kirpal Bagh, Gujranwala Town, New Delhi, Delhi, 110033)

(2024-2027)

CERTIFICATE

This is to certify that the Summer Training Report entitled "Loan Default Risk Predictor: An Explainable Machine Learning Web Application for Credit Risk Assessment" is a bonafide work carried out by GURSIMRAN KAUR, Enrolment No. 02490202024, a student of Bachelor of Computer Applications (BCA), Semester V, Section 5A, at Sri Guru Tegh Bahadur Institute of Management and Information Technology, in partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Applications.

The work was carried out under my guidance during the six-week summer training, and to the best of my knowledge it is original and has not been submitted elsewhere for any degree or diploma.

Ms. Nisha Choudhary
Mentor / Guide

Date: _____ Place: _____

COMPANY'S CERTIFICATE

Certificate of Completion issued by GRAStech to GURSIMRAN KAUR for successfully completing the six-week online internship program in Machine Learning Using Python. Date of issue: 31 July 2026. Signed by Mohammad Qayam Ali, Founder and Program Head.



CANDIDATE'S DECLARATION

I, GURSIMRAN KAUR, Enrolment No. 02490202024, a student of BCA, Semester V, Section 5A, Sri Guru Tegh Bahadur Institute of Management and Information Technology, hereby declare that the Summer Training Report entitled "Loan Default Risk Predictor: An Explainable Machine Learning Web Application for Credit Risk Assessment" is my own original work, carried out during the six-week online internship in Machine Learning Using Python at GRAStech, and that it has been written in my own words.

I further declare that this report has not been copied from any book, website or another student, that wherever the work of others has been used it has been acknowledged in the references, and that this report has not been submitted to any other institute or university for the award of any degree or diploma.

GURSIMRAN KAUR
Enrolment No.: 02490202024

Date: _____ Place: _____

ACKNOWLEDGEMENT

I express my sincere gratitude to everyone who helped me in completing this Summer Training Report and the project Loan Default Risk Predictor.

I am thankful to GRAStech and Mohammad Qayam Ali, Founder and Program Head, for providing the opportunity of a six-week online internship in Machine Learning Using Python, which gave me the practical exposure needed to build this project.

I am grateful to my mentor, Ms. Nisha Choudhary, Assistant Professor, for guiding me, reviewing the report and the working project, and giving valuable suggestions at every stage.

I thank the management and faculty of Sri Guru Tegh Bahadur Institute of Management and Information Technology for providing a good learning environment.

GURSIMRAN KAUR

ABSTRACT

Lenders that extend consumer credit must decide, quickly and consistently, whether an applicant is likely to repay a loan, in a way that is explainable to the applicant and defensible to a regulator. Manual underwriting varies between officers, traditional scorecards struggle to capture non-linear effects among income, loan amount and credit history, and the machine learning models that do capture such effects usually behave as a black box, giving a probability with no reason a loan officer can examine. A missed default and a wrongly declined applicant also carry different costs, and few tools let an institution weigh the two against each other.

Loan Default Risk Predictor is a web application, developed during the six-week summer training (an online internship in Machine Learning Using Python at GRAStech), that scores an applicant's probability of default and explains the score factor by factor. A user enters twelve fields covering the applicant, the loan and the existing credit position, or loads a sample file, and receives a calibrated probability of default, a risk band, a score and a lending recommendation, along with a waterfall of the contribution each factor made to the score. A what-if control lets the user move the approval cutoff and see how the approval rate, the bad rate and the expected cost change.

The model is a gradient boosting ensemble of 250 trees, trained on the UCI German Credit dataset of 1,000 applicants, benchmarked against logistic regression, random forest and XGBoost under five-fold cross-validation, tuned by randomized search and Platt-calibrated so its output reads as a true probability. Contributions are computed with a Saabas tree-path decomposition, rescaled to sum exactly to the predicted probability. The application runs Next.js and React on the front end and a dependency-free Python scoring function on the back end, deployed as a Vercel serverless function, and was verified with 13 notebook tests, 25 API tests and 18 interface tests, all passing.

The report covers the problem formulation, system analysis, system design including the ERD, DFD and use case diagrams, the model's evaluation, testing, implementation, limitations, future scope, and annexures of screen snapshots and source code. The live application is available at https://credoscore-gk.vercel.app.

Keywords: Credit Risk, Probability of Default, Gradient Boosting, Platt Scaling, Explainable AI, Next.js, Python, Vercel.

LIST OF FIGURES

Figure 2.1 Project schedule of the six-week training 8
Figure 3.1 Three-tier architecture of the Loan Default Risk Predictor 15
Figure 3.2 Module hierarchy of the Loan Default Risk Predictor 16
Figure 3.3 Entity Relationship Diagram of the Loan Default Risk Predictor 18
Figure 3.4 Context Level DFD (Level 0) 22
Figure 3.5 Level 1 DFD 22
Figure 3.6 Use case diagram 24
Figure 3.7 Flowchart of the training process 25
Figure 3.8 Flowchart of the scoring process 26
Figure 3.9 State diagram of an applicant's decision 26
Figure 3.10 Cross-validated ROC-AUC of the candidate models 32
Figure 3.11 Reliability diagram of the calibrated probabilities 34
Figure 3.12 ROC curve of the calibrated model 35
Figure 3.13 Feature importance of the twelve retained fields 36
Figure 3.14 Observed versus predicted default rate by decile 36
Figure 3.15 Approval rate, bad rate and cost per applicant against the cutoff 38
Figure 4.1 Deployment of the Loan Default Risk Predictor 50
Figure A-1 Blank applicant assessment form 57
Figure A-2 A sample file loaded through a preset chip 58
Figure A-3 A field-level validation error on submit 58
Figure A-4 The what-if cutoff control moved to 40 percent 59
Figure A-5 The landing page 59
Figure A-6 A low-risk result slip 60
Figure A-7 A high-risk result slip 60
Figure A-8 The score-drivers waterfall, close up 61
Figure A-9 The portfolio band split and cutoff explorer 61
Figure A-10 The portfolio decile view 62
Figure A-11 The model benchmark on the methodology page 62
Figure A-12 The calibration curve on the methodology page 63
Figure A-13 The assessment workbench on a mobile screen 63
Figure A-14 The landing page in dark mode 64

LIST OF TABLES

Table 1.1 Comparison of existing approaches and the proposed system 2
Table 2.1 Iterations followed during development 8
Table 2.2 Actors and their responsibilities 9
Table 2.3 Functional requirements 10
Table 2.4 Role-permission matrix 12
Table 2.5 Software used 12
Table 2.6 Hardware used 13
Table 3.1 Folder and file structure 16
Table 3.2 Structure of the training dataset (fields used) 18
Table 3.3 Encoding of categorical fields 19
Table 3.4 Structure of model.json 20
Table 3.5 Description of Level 1 processes 23
Table 3.6 Use case description: Assess an applicant 24
Table 3.7 Applicant, loan and credit position sections 27
Table 3.8 Sample files and what-if controls 27
Table 3.9 REST API endpoints 29
Table 3.10 Prediction response fields 29
Table 3.11 Cross-validated benchmark of candidate models 32
Table 3.12 Search space and selected values 33
Table 3.13 Out-of-fold evaluation metrics 34
Table 3.14 Confusion matrix at the 0.25 cutoff 35
Table 3.15 Approval, bad rate and cost by cutoff 37
Table 4.1 Model fidelity and correctness tests 41
Table 4.2 API validation and robustness tests 42
Table 4.3 Model quality and performance tests 44
Table 4.4 User interface tests 45
Table 4.5 Endpoint and integration tests 46
Table 4.6 Test summary 47
Table 4.7 Defects found and corrections made 48
Table 4.8 Using the system 50

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
AUC	Area Under the Curve
BCA	Bachelor of Computer Applications
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
CV	Cross-Validation
DFD	Data Flow Diagram
DM	Deutsche Mark
ERD	Entity Relationship Diagram
FN	False Negative
FP	False Positive
GB	Gradient Boosting
JSON	JavaScript Object Notation
KS	Kolmogorov-Smirnov (statistic)
ML	Machine Learning
OOF	Out-of-Fold
PD	Probability of Default
PR-AUC	Precision-Recall Area Under the Curve
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
STR	Summer Training Report
UCI	University of California, Irvine
UI	User Interface
XAI	Explainable Artificial Intelligence

C O N T E N T S

CHAPTER 1: PROBLEM FORMULATION **1**

1.1 Introduction about the Company 1

1.2 Introduction about the Problem 1

1.3 Present State of the Art 2

1.4 Need of Computerization 3

1.5 Proposed Software / Project 4

1.6 Importance of the Work 5

CHAPTER 2: SYSTEM ANALYSIS **7**

2.1 Feasibility Study 7

2.2 Analysis Methodology 8

2.3 Choice of the Platforms 12

CHAPTER 3: SYSTEM DESIGN **15**

3.1 Design Methodology 15

3.2 Database Design 18

3.3 Input Design 26

3.4 Output Design 28

3.5 Code Design and Development 28

3.6 Model Design and Development 31

CHAPTER 4: TESTING AND IMPLEMENTATION **40**

4.1 Testing Methodology 40

4.2 Test Data & Test Cases 41

4.3 Test Reports and Debugging 47

4.4 Implementation 49

CHAPTER 5: CONCLUSION AND REFERENCES **52**

5.1 Summary of the Work Done 52

5.2 Conclusion 52

5.3 Limitations of the System 53

5.4 Future Scope for Modification 54

5.5 References / Bibliography 55

CHAPTER 6: ANNEXURES **57**

A-1 Sample Inputs 57

A-2 Sample Outputs 59

A-3 Coding 64

CHAPTER 1

PROBLEM FORMULATION

Learning Objectives

- Understand the training background and the company behind the internship
- Identify the problem lenders face in assessing credit risk
- Study the present state of the art and the need for computerisation
- Define the objectives, scope and importance of the Loan Default Risk Predictor

CHAPTER 1: PROBLEM FORMULATION

1.1 Introduction about the Company

GRAStech is an organisation that runs online internship programs in technology fields. The program undertaken by the trainee was the six-week online internship program in Machine Learning Using Python. The program is headed by Mohammad Qayam Ali, Founder and Program Head.

The compulsory six-week summer training was completed as an online internship in Machine Learning Using Python at GRAStech, and the company issued a Certificate of Completion dated 31 July 2026, signed by Mohammad Qayam Ali. A copy of this certificate is placed after the Institute's Certificate at the beginning of this report. The training was carried out under the guidance of the college mentor, Ms. Nisha Choudhary, Assistant Professor. During the internship the trainee learned the complete life cycle of a machine learning application: data understanding, data preparation, modelling, evaluation and deployment, along with the testing that every stage requires. The project titled Loan Default Risk Predictor was designed and developed individually by the trainee as the training project.

1.2 Introduction about the Problem

A lender that approves an unsecured loan is taking on a risk that is settled only months or years later, when the applicant has either repaid the loan or defaulted on it. The decision to approve, decline or ask for conditions has to be made at the time of application, from the information the applicant has provided and the lender's own records, and it has to be consistent across every officer who makes it. Where that decision is left entirely to individual judgement, two applicants with a similar credit history can receive different outcomes depending on who reviewed the file, and the reasoning behind a decline is rarely written down in a form the applicant can be shown.

A default is costly to the lender in a way that a cautious decline is not. Every unit of loan that is extended to an applicant who later defaults has to be weighed against the much smaller cost of asking a genuinely creditworthy applicant to wait or to provide additional conditions, so the two kinds of mistakes are not equally expensive and a sound decision rule has to account for that difference rather than treat every error alike. Statistical scorecards have been used for decades to make this judgement more consistent, but a scorecard built on a simple weighted sum of the applicant's attributes does not capture the way several factors interact, for instance

how a short employment history matters far more when it is combined with a long loan duration than on its own. Machine learning models that are flexible enough to capture such interactions exist, but most are used as a black box: the model returns a number, and neither the applicant nor the loan officer can see which factors pushed the score up or down, which makes the decision difficult to explain, to audit or to contest.

The problem can therefore be summarised in one statement: there is no simple, free and explainable tool that turns an applicant's details into a calibrated probability of default, a lending recommendation and a factor-by-factor reason for that recommendation, in a way a loan officer can defend and an applicant can understand.

1.3 Present State of the Art

Credit risk assessment today is carried out through a mix of approaches. Purely manual, expert-judgement underwriting relies on an officer's experience and a checklist of documents, and remains common for small lenders, but it does not scale and is hard to make consistent. Traditional logistic-regression scorecards, of the kind described by Siddiqi, assign points to binned attributes and sum them into a score; they are easy to explain because each bin contributes a fixed number of points, but they are built around a linear combination of the inputs and need manual binning to approximate non-linear effects. Credit bureau scores aggregate a borrower's history across institutions and are useful where that history exists, but they are an external, general-purpose number rather than something a lender can tune to its own cost of a bad loan. Black-box machine learning models, typically gradient boosting or random forest ensembles, are now used by many larger lenders because they handle non-linear and interaction effects well, but deployed without a calibration step or an explanation layer, their raw output is an uninterpreted score rather than a probability, and individual decisions cannot easily be justified.

A comparison of these approaches with the proposed system is given in Table 1.1.

Table 1.1 Comparison of existing approaches and the proposed system

Capability	Manual underwriting	Traditional scorecard	Black-box ML	Proposed system
Probability output	No	Yes	Yes	Yes
Calibrated	No	Partial	Rarely	Yes

Capability	Manual underwriting	Traditional scorecard	Black-box ML	Proposed system
Explanation per applicant	Varies by officer	Yes	No	Yes
Handles non-linear effects	No	No	Yes	Yes
Cost-aware cutoff	No	Partial	Rarely	Yes
Deployment without a heavy runtime	Yes	Yes	No	Yes
Free and open	Varies	Varies	Varies	Yes

The table shows that no single existing approach combines a calibrated probability, a per-applicant explanation, non-linear modelling and a cost-aware cutoff, all deployed without a heavy runtime and at no licensing cost. The Loan Default Risk Predictor is designed to occupy that space.

1.4 Need of Computerization

A computerised, explainable system is needed because the present mix of manual and black-box approaches suffers from the following drawbacks:

- Manual decisions are inconsistent, since two officers can read the same application differently.
- A plain score, whether from a scorecard or a black-box model, gives no reason the applicant or the officer can inspect.
- Black-box models are rarely checked for calibration, so a quoted probability of default may not match the true default rate at that score.
- The cost of a missed default and the cost of a wrongly declined applicant are different, and few manual or black-box processes make that trade-off explicit or adjustable.
- There is no easy way for an officer to ask "what would change if the cutoff moved", since re-running a scorecard or a black-box model by hand for every cutoff is impractical.
- Portfolio-level questions, such as how the bad rate is split across risk bands or whether the model ranks applicants well, are difficult to answer without a system that logs and aggregates every scored application.

A computerised and explainable system removes these drawbacks. It applies the same calibrated model to every applicant, produces a probability and a factor-by-factor reason for it in a fraction of a second, lets the cutoff be moved and its consequences inspected instantly, and aggregates every historical prediction into the portfolio-level view a risk analyst needs.

1.5 Proposed Software / Project

The Loan Default Risk Predictor is a web application that scores a loan applicant's probability of default with a calibrated gradient boosting model and explains the score factor by factor. It is a three-tier application consisting of a Next.js and React front end, a dependency-free Python scoring function deployed as a Vercel serverless function, and two JSON artifacts produced by an offline training pipeline. The live application is available at https://credoscore-gk.vercel.app.

1.5.1 Objectives of the Project

1. To collect the twelve applicant, loan and credit-position fields needed to score a loan application, with client-side and server-side validation.
2. To train and benchmark a gradient boosting model against logistic regression, random forest and XGBoost, and to calibrate its output into a genuine probability of default.
3. To return, for every scored applicant, a probability of default, a risk band, a credit-style score and a lending recommendation.
4. To explain every prediction with a factor-by-factor contribution that sums exactly to the predicted probability.
5. To let a user move the approval cutoff and see the resulting approval rate, bad rate among approved applicants and expected cost change in real time.
6. To provide portfolio-level analytics, namely the split of applicants by risk band, the cutoff trade-off, the decile-wise lift and the default rate by segment.
7. To deploy the system on a free hosting platform so that it can be accessed publicly through a web link, without requiring a heavy machine learning runtime at request time.

1.5.2 Scope of the Project

The scope of the project covers the following:

- Capturing the twelve fields used by the model and validating them on both the browser and the server.
- Loading one of several sample applicant files to fill the form quickly.
- Scoring an applicant and showing the probability of default, the risk band, the score and the recommendation.
- Explaining the score with a waterfall of individual factor contributions.
- A what-if control that recomputes the band, the score and the decision as the cutoff is moved.
- Portfolio analytics: band split, cutoff trade-off, deciles and segments.
- A methodology page describing the model, its benchmark against other candidate models, its calibration and its known limitations.
- A JSON REST API exposing the same scoring and portfolio functionality independently of the web interface.

The following are outside the present scope: using the tool for real lending decisions, a formal fairness audit of the model against protected characteristics, user accounts, storing submitted applications, and retraining the model from within the user interface. These are listed as future enhancements in Chapter 5.

1.6 Importance of the Work

The work is important on three counts. It tackles a problem that sits at the centre of consumer lending, namely turning an applicant's details into a decision that is both accurate and explainable, so the requirements were grounded in a real, well-studied dataset rather than an invented scenario. The project is also original and was built from the training data upward: the benchmarking, the tuning, the calibration and the explanation method were all carried out by the trainee rather than taken from an existing library untouched, giving the result genuine value as a teaching example of an explainable scoring system. It also provided practical exposure to the full life cycle of an applied machine learning product, covering feature engineering, model benchmarking and tuning, probability calibration, explanation, a dependency-free production API, a front end, testing and cloud deployment.

The project also demonstrates professional practices expected of a developer working on a credit model. The personal-status-and-sex attribute in the source data was excluded as a protected characteristic before any modelling began. The model's output was calibrated and

evaluated honestly on out-of-fold predictions rather than on the data it was trained on. No applicant data is retained by the deployed API, which is stateless and keeps no secrets, and the report states plainly, in every place the tool is shown, that it is not meant for real lending decisions.

CHAPTER 2

SYSTEM ANALYSIS

Learning Objectives

- Assess the feasibility of the project
- Describe the analysis methodology and the requirements
- Identify actors and permissions
- Justify the choice of software and hardware

CHAPTER 2: SYSTEM ANALYSIS

2.1 Feasibility Study

Before development began, a feasibility study was carried out to check whether the proposed system could be built within the available time and skills, and whether it would be useful in practice.

Technical Feasibility

All the technologies chosen are open source, well documented and widely used: Next.js and React for the front end, Python's standard library for the scoring function, and pandas, scikit-learn and XGBoost for training. The trainee had the required background in JavaScript, TypeScript and Python, and received guidance during the training for the modelling and calibration steps. The German Credit dataset is small enough to train on an ordinary laptop in under a minute, and Vercel offers a free tier that auto-detects Next.js and deploys Python functions alongside it. The project is therefore technically feasible.

Economic Feasibility

The cost of the system is effectively zero. The dataset is licensed CC BY 4.0 and carries no fee, every library used is open source, development was carried out on an ordinary laptop, and hosting on Vercel's free tier is sufficient for a project of this size. The benefit, in the form of a faster and more consistent credit decision, outweighs this negligible cost, so the project is economically feasible.

Operational Feasibility

The interface follows the pattern of an ordinary web form: a set of labelled fields, a submit button and a results panel. A loan officer does not need training to fill in the twelve fields or to read a probability, a band and a recommendation. A risk analyst reads the portfolio pages the same way as any dashboard. The system therefore fits naturally into how its three user roles already work, so operational acceptance is expected to be high.

Schedule Feasibility

The training period was six weeks. The work was divided into six weekly iterations that could be developed and tested one after another, as shown in Figure 2.1. The schedule was realistic and the project was completed within the training period.

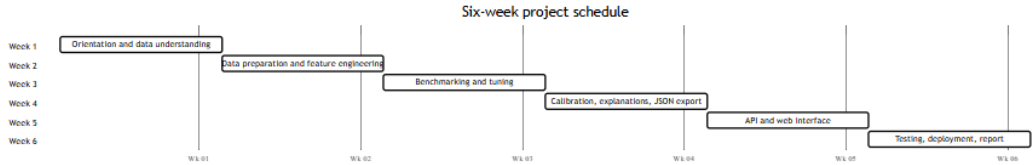


Figure 2.1 Project schedule of the six-week training

Legal and Security Feasibility

The dataset used, the UCI Statlog German Credit Data, is licensed under CC BY 4.0 and attributed to Professor Hans Hofmann of Universität Hamburg, so its use in this project is legally clear. The personal-status-and-sex attribute of the original dataset is excluded from the model as a protected characteristic, and the deployed application does not store any applicant data it is given; every request to the scoring API is stateless. The report and the application state throughout that the tool is a student project and is not meant for real lending decisions.

2.2 Analysis Methodology

An iterative, CRISP-DM-style approach was followed, in which data understanding, data preparation, modelling, evaluation and deployment were revisited in successive iterations rather than attempted once from end to end. Each iteration produced a working piece of the system, which was tested before the next iteration began, as summarised in Table 2.1.

Table 2.1 Iterations followed during development

Iteration	Work carried out	Result
1	Orientation and data understanding	The German Credit dataset, its attributes and its licence were studied, and the attributes to exclude were decided
2	Data preparation and feature engineering	Employment buckets, derived monthly income and one-hot encoding of categorical fields were implemented
3	Benchmarking and tuning	Four candidate models were compared under five-fold cross-validation and the gradient boosting model was tuned by randomized search
4	Calibration, explanation and export	Platt scaling, the Saabas contribution rescaling and the model.json / portfolio.json export were implemented

Iteration	Work carried out	Result
5	API and web interface	The Python scoring and portfolio functions and the Next.js front end were built
6	Testing, deployment and report	Notebook, API and interface tests were run, the application was deployed, and this report was written

2.2.1 Requirement Gathering

Requirements were gathered by studying how credit scorecards and bureau scores are used in practice, by working through the UCI German Credit dataset's own documentation to see what information is realistically available at the point of application, and by reviewing the requirement with the training mentor before design started. The outcome was the functional and non-functional requirements set out below.

2.2.2 Users of the System

The system is used by three actors, whose responsibilities are set out in Table 2.2.

Table 2.2 Actors and their responsibilities

Actor	Responsibility
Loan officer	Enters an applicant's details or loads a sample file, reads the probability of default, the score and the recommendation, reviews the factor-by-factor explanation, and moves the what-if cutoff for an individual applicant.
Risk analyst	Reads the portfolio-level analytics (band split, cutoff trade-off, deciles, segments) and the methodology page to judge whether the deployed cutoff and the model remain appropriate.
Model maintainer	Retrains the model on updated data, keeps the exported model.json and portfolio.json artifacts in sync with the decision thresholds in decision.ts, and is responsible for the model's version number.

2.2.3 Functional Requirements

The requirements gathered from the dataset, the training mentor and the trainee's own study of credit scoring were consolidated into the fourteen functional requirements listed in Table 2.3.

Table 2.3 Functional requirements

ID	Requirement	Actor
FR1	The system shall capture the twelve required applicant, loan and credit-position fields, with client-side and server-side validation of numeric bounds, known categories and the instalment cross-field rule.	Loan officer
FR2	The system shall allow a sample applicant file to be loaded to pre-fill the form.	Loan officer
FR3	The system shall return, for every submitted applicant, a probability of default, a risk band, a credit-style score and a lending recommendation.	Loan officer
FR4	The system shall show a factor-by-factor contribution waterfall whose values sum exactly to the predicted probability of default.	Loan officer
FR5	The system shall allow the approval cutoff to be moved in a what-if control and shall recompute the band, score and decision accordingly.	Loan officer
FR6	The system shall show a portfolio-level split of applicants by risk band.	Risk analyst
FR7	The system shall let the user explore the approval rate, the bad rate among approved applicants and the expected cost across a grid of cutoffs.	Risk analyst
FR8	The system shall show the model's decile-wise observed default rate against its predicted rate.	Risk analyst
FR9	The system shall show the default rate across applicant segments.	Risk analyst
FR10	The system shall provide a methodology page describing the model, its benchmark against other candidate models, its calibration and its known limitations.	Risk analyst
FR11	The system shall allow the model maintainer to retrain the model and export it as a versioned JSON artifact.	Model maintainer
FR12	The system shall show field-level validation errors on the form, with `aria-invalid` attributes and focus moved to the first invalid field.	Loan officer

ID	Requirement	Actor
FR13	The system shall expose the scoring and portfolio functionality through a JSON REST API with CORS enabled, usable independently of the web interface.	Model maintainer
FR14	The system shall provide a light and dark theme, selectable by the user, and a layout usable from 360px of viewport width upward.	Loan officer

2.2.4 Non-Functional Requirements

- Security: strict server-side validation of every field, no stack traces returned to the client, a stateless API that stores no applicant data.
- Usability: field-level error messages, sample files to try the system quickly, a layout that works on a phone screen.
- Performance: the scoring function returns a result in well under a second; the underlying tree-walk and explanation computation measured a mean latency of 0.729 ms and a p95 of 0.907 ms over 1,000 calls.
- Reliability: invalid input must never crash the scoring function; every request returns a 2xx or a 400 response with a clear message, and there are no 500 errors.
- Portability: the front end works in every modern browser and on mobile screens, and the scoring function has no runtime dependency beyond the Python standard library.
- Maintainability: the decision thresholds are kept in one place on each side (decision.ts on the front end, predict.py on the back end) so that a change to the cutoff logic can be made and checked in both places together.
- Accessibility: labelled form fields, visible focus states, aria-invalid on failed fields, and colours for the risk bands chosen to be distinct from the brand colour and from each other.

2.2.5 Role-Permission Matrix

The permissions granted to each actor over every operation are set out in Table 2.4.

Table 2.4 Role-permission matrix

Operation	Loan officer	Risk analyst	Model maintainer
Submit an applicant for scoring	Yes	Yes	Yes
View the score and the factor explanation	Yes	Yes	Yes
Adjust the what-if cutoff	Yes	Yes	Yes
View portfolio analytics	View only	Yes	Yes
View the methodology and limitations page	View only	Yes	Yes
Call the JSON API directly	No	No	Yes
Retrain and export the model	No	No	Yes
Change the decision thresholds	No	No	Yes

2.3 Choice of the Platforms

The technologies were selected after comparing the alternatives against four criteria: cost, suitability for a small tabular dataset and an explainable model, fit with a stateless serverless deployment, and community support.

2.3.1 S/W used

The software stack chosen for the training pipeline, the API and the web interface is listed in Table 2.5.

Table 2.5 Software used

Component	Technology	Reason for choice
Front-end framework	Next.js 16.3.5 (App Router, Turbopack)	Server-rendered React with file-based routing and a fast development server.
UI library	React 19.2.8	Component model shared by the application's client islands.
Language	TypeScript 5	Static typing catches shape errors in the scoring pipeline early.
Styling	Tailwind CSS 4	Utility classes keep the editorial theme consistent without a separate design-system package.
UI primitives	Radix UI	Accessible, unstyled primitives for the form and overlay components.

Component	Technology	Reason for choice
Charting	Recharts 3	Declarative charts for the waterfall, band split and cutoff explorer.
Motion	Framer Motion	Restrained reveal and staircase-bar animations.
Icons	lucide-react	A consistent icon set for the navigation and the form.
Scoring runtime	Python standard library (api/predict.py, api/portfolio.py)	A dependency-free handler keeps the deployed function small and avoids shipping scikit-learn.
Data handling (training)	pandas 2.2.3, numpy 2.1.3	Data loading, feature engineering and array computation during training.
Modelling (training)	scikit-learn 1.9.1, xgboost 2.1.3	The candidate models compared under cross-validation.
Explanation cross-check	shap 0.46.0	An independent check of the Saabas contributions used in production.
Training reporting	matplotlib 3.9.2, jupyter	Notebook-based benchmarking, evaluation charts and test documentation.
Hosting	Vercel	Auto-detects the Next.js application and deploys api/*.py as Python serverless functions.

2.3.2 H/W used

The hardware used for development, and the minimum hardware expected at the client end, is listed in Table 2.6.

Table 2.6 Hardware used

Component	Specification
Processor	Intel Core i5 or equivalent (or higher)
RAM	8 GB minimum, 16 GB recommended
Storage	256 GB SSD, at least 2 GB free
Display	Standard laptop or desktop monitor (1366 x 768 or higher)
Network	Broadband internet connection
Client device (end user)	Any computer, tablet or smartphone with a modern web browser

2.3.3 Software Requirement Summary

To run the project locally, Node.js, Python 3.12, npm and a code editor are needed, along with the training dependencies (pandas, numpy, scikit-learn, xgboost, shap, matplotlib, jupyter) if the model is to be retrained. The development setup runs two local servers, the Next.js application and the Python prediction and portfolio servers, with the Next.js configuration rewriting requests to them in development only. For the end user, only a browser and an internet connection are required.

CHAPTER 3

SYSTEM DESIGN

Learning Objectives

- Explain the system's architecture, module hierarchy and project structure
- Design the data structures and process models: ERD, DFD, use case diagram and flowcharts
- Describe the input design, the output design, the REST API and the security design
- Describe how the scoring model was selected, tuned, calibrated, evaluated and explained

CHAPTER 3: SYSTEM DESIGN

3.1 Design Methodology

The system was designed top-down: architecture first, then module boundaries, then folder structure, then data design, then screens. Structured design tools were used throughout: an ERD for the data the system reads and writes, DFDs for the movement of a request through the scoring pipeline, a use case diagram for actor-system interaction, and flowcharts and a state diagram for training and scoring.

3.1.1 System Architecture

The Loan Default Risk Predictor follows a three-tier architecture, shown in Figure 3.1. The browser never evaluates the model itself; every scoring request is sent to a server-side function, and every reusable number used to build a page is read from a JSON artifact at build time.

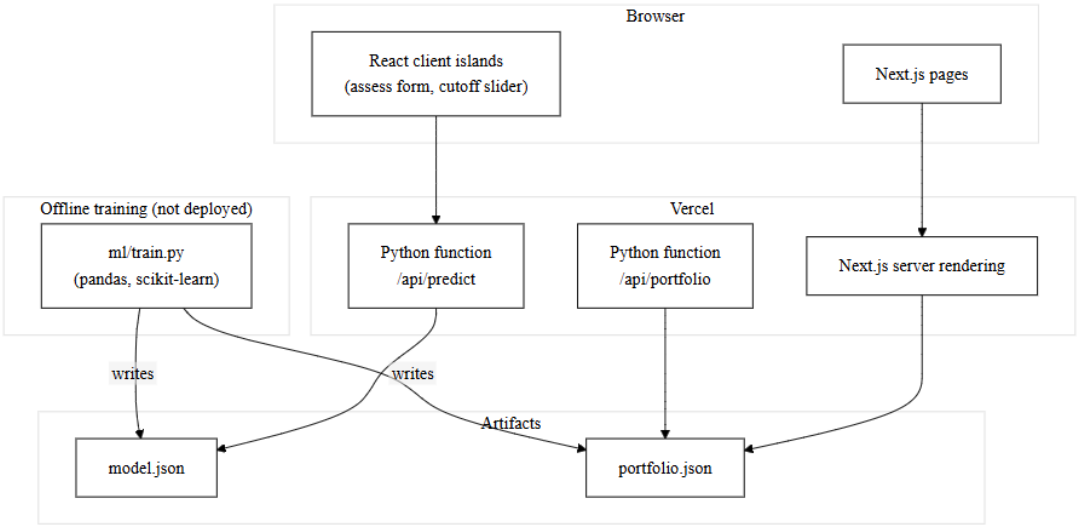


Figure 3.1 Three-tier architecture of the Loan Default Risk Predictor

- **Presentation layer.** Next.js 16 App Router pages (Brief, Assess, The book, Methodology) rendered on the server and hydrated as React 19 client islands only where interaction is required, for example the applicant form and the cutoff slider.
- **Application layer.** Vercel hosts both the Next.js server, which renders pages and serves static assets, and two standard-library Python functions, `api/predict.py` and

`api/portfolio.py`, which are the only parts of the system the browser calls over the network at runtime.

- **Data layer.** Two JSON artifacts, `model/model.json` and `model/portfolio.json`, hold the fitted ensemble and the precomputed portfolio aggregates respectively. A separate offline path, `ml/train.py`, reads the raw dataset with pandas and scikit-learn and writes both artifacts; it never runs in production.

3.1.2 Module Hierarchy

The system divides into four modules, shown in Figure 3.2: the training pipeline, the scoring API, the web interface and the test suite. The training pipeline prepares data, benchmarks candidate models, tunes and calibrates the winner and exports the two artifacts. The scoring API validates a request, walks the tree ensemble, calibrates and bands the result, explains it and serves the portfolio analytics. The web interface covers the four pages reachable from the navigation bar. The test suite is the notebook's model tests together with the API and UI test scripts described in Chapter 4.



Figure 3.2 Module hierarchy of the Loan Default Risk Predictor

3.1.3 Project Structure

The repository's folder and file structure is summarised in Table 3.1, verified against the project tree after the user interface was rebuilt.

Table 3.1 Folder and file structure

File / Folder	Purpose
`app/layout.tsx`, `app/page.tsx`, `app/globals.css`	Root layout, Brief landing page, global stylesheet.
`app/assess/`, `app/portfolio/`, `app/model/`	The Assess, The book and Methodology pages.
`app/error.tsx`, `app/not-found.tsx`	Route-level error and 404 screens.

File / Folder	Purpose
components/assess/	Applicant form, workbench, contribution waterfall, cutoff control, PD meter, verdict stamp, sample-file presets.
components/portfolio/	Risk-band split, cutoff explorer, decile chart/table, segment explorer.
components/model/	Benchmark table, calibration chart, cutoff table, importance bars, ROC chart.
components/landing/	Calibration miniature, cost-asymmetry panel, decile staircase, decision pipeline diagram.
components/site/, components/editorial/	Nav, footer, theme toggle; and shared layout primitives (section, stat, figure, reveal).
components/ui/, components/charts/chart-kit.tsx	Radix-based primitives and the shared Recharts configuration.
lib/types.ts, lib/format.ts, lib/utils.ts	Shared types, label/number formatters, the cn() helper.
lib/decision.ts, lib/portfolio.ts, lib/model-meta.ts	Client decision mirror, portfolio reader, server-only model metadata reader.
lib/api.ts	The typed predict() client for /api/predict.
api/predict.py, api/portfolio.py	The two Python functions deployed as Vercel serverless functions.
ml/train.py, ml/dev_server.py	Offline training pipeline; local dev server mirroring the two functions on ports 8001/8002.
model/model.json, model/portfolio.json	Exported ensemble and precomputed portfolio aggregates.
data/german_credit.csv	The cleaned, re-labelled training dataset.
notebook.ipynb	Reproducibility and quality-gate tests (Chapter 4).
docs/	Internal engineering notes (architecture, data, model, testing, deployment, limitations).
vercel.json, next.config.ts	Function bundling config and the local rewrite to the two Python servers.

3.2 Database Design

The system has no transactional database. Its data design covers the training dataset and the two JSON artifacts the API reads.

3.2.1 ERD

An applicant submits one loan application, which carries exactly one credit position and receives exactly one prediction from the deployed model artifact; a prediction is explained by one or more per-factor contributions. The relationships are drawn in Figure 3.3.

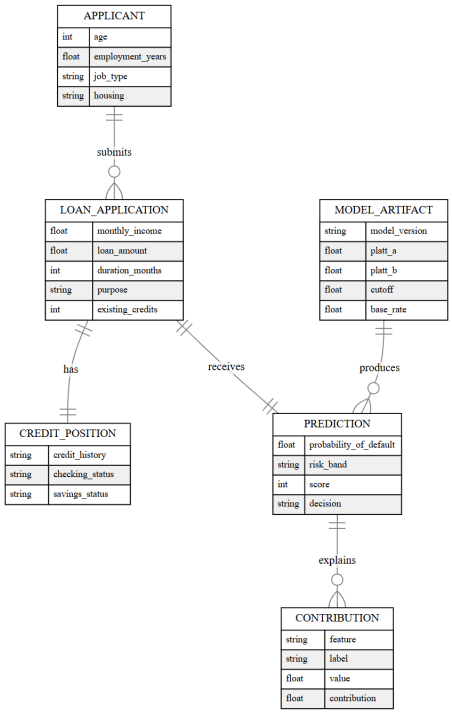


Figure 3.3 Entity Relationship Diagram of the Loan Default Risk Predictor

Collection Structure

The training dataset's twelve retained fields are structured as shown in Table 3.2.

Table 3.2 Structure of the training dataset (fields used)

Field	Type	Values / range	Source attribute
age	Numeric	18–100 years	Attribute 13
employment_years	Numeric	0, 0.5, 2.5, 5.5, 10 (bucket midpoints)	Attribute 7 (A71–A75)

Field	Type	Values / range	Source attribute
monthly_income	Numeric	1–1,000,000 DM	Engineered from Attributes 2, 5 and 8
loan_amount	Numeric	1–1,000,000 DM	Attribute 5
duration_months	Numeric	1–120 months	Attribute 2
existing_credits	Numeric	0–20	Attribute 16
job_type	Categorical	management, skilled, unskilled_nonresident, unskilled_resident	Attribute 17
housing	Categorical	free, own, rent	Attribute 15
purpose	Categorical	business, car_new, car_used, domestic_appliances, education, furniture_equipment, other, radio_tv, repairs, retraining	Attribute 4
credit_history	Categorical	all_paid, critical, delayed_past, no_credits, paid_duly	Attribute 3
checking_status	Categorical	0_to_200, 200_or_more, less_than_0, none	Attribute 1
savings_status	Categorical	100_to_500, 500_to_1000, none, over_1000, under_100	Attribute 6

monthly_income does not exist as a UCI attribute. It is engineered at load time as `(loan_amount / duration_months) / (installment_rate / 100)`, so that a 1,169 DM loan over 6 months at an installment rate of 4% implies a monthly income of 4,870.83 DM. Attribute 9 (personal status and sex) is excluded as a protected attribute, and Attributes 10, 11, 12, 14, 18, 19 and 20 are excluded to keep the applicant form short.

Each of the six categorical fields is one-hot encoded; Table 3.3 gives the resulting column counts.

Table 3.3 Encoding of categorical fields

Field	Categories	One-hot columns
job_type	4	4
housing	3	3

Field	Categories	One-hot columns
purpose	10	10
credit_history	5	5
checking_status	4	4
savings_status	5	5

The six numeric fields pass through unscaled, so the full feature vector that the ensemble sees has 6 + 31 = 37 columns. The A47 vacation purpose code is absent from the data entirely, and A48 (retraining) is present in only 9 of the 1,000 rows.

`model/model.json`, the deployed artifact, is structured as shown in Table 3.4.

Table 3.4 Structure of model.json

Key	Type	Description
model_version	String	Semantic version of the exported artifact ("1.0.0").
model_type	String	"gradient_boosting".
trained_on_rows	Integer	1,000.
numeric_features, categorical_features	Array of String	The six numeric and six categorical field names, in vector order.
categories	Object	Category list per categorical field, in encoding order.
learning_rate	Number	0.05.
base_value	Number	The ensemble's starting log-odds, ln(0.3 / 0.7).
platt_a, platt_b	Number	The Platt scaling coefficients fitted on out-of-fold margins.
trees	Array of Object	250 trees, each with left, right, feature, threshold and value arrays.
base_rate	Number	0.30, the dataset's observed default rate.
cutoff	Number	0.25, the deployed approval cutoff.
metrics, benchmark	Object	Out-of-fold evaluation metrics and the cross-validated model comparison.
hyperparameters	Object	The selected values from the randomised search.

Validations, Verifications and Normalization

- Field-level validation: the API checks that all twelve fields are present, that the six numeric fields are finite numbers within their documented bounds, and that the six categorical fields hold a value from the trained category list; any failure is rejected before scoring.
- Cross-field validation: the implied monthly instalment, loan_amount divided by duration_months, may not exceed four times monthly_income.
- Parity check: at export time, the hand-written tree walk in `api/predict.py` is compared against the fitted scikit-learn pipeline's output, and the exported artifact is accepted only when the maximum difference across the training rows is below 1e-9.
- Additivity check: the per-factor contributions produced during explanation are summed and compared against the probability minus the base rate; the notebook asserts this difference stays below 1e-14.
- Encoding normalisation: an unseen category in the request is rejected outright rather than silently mapped to a default, since the one-hot scheme has no "unknown" column.
- Privacy: the protected attribute (personal status and sex) is excluded from training altogether, and no applicant's input or output is persisted anywhere; the API is stateless.

3.2.2 DFD

Figure 3.4 is the context-level diagram (Level 0), which treats scoring and analytics as a single process bounded by three external entities. Figure 3.5 breaks that process into six processes at Level 1.

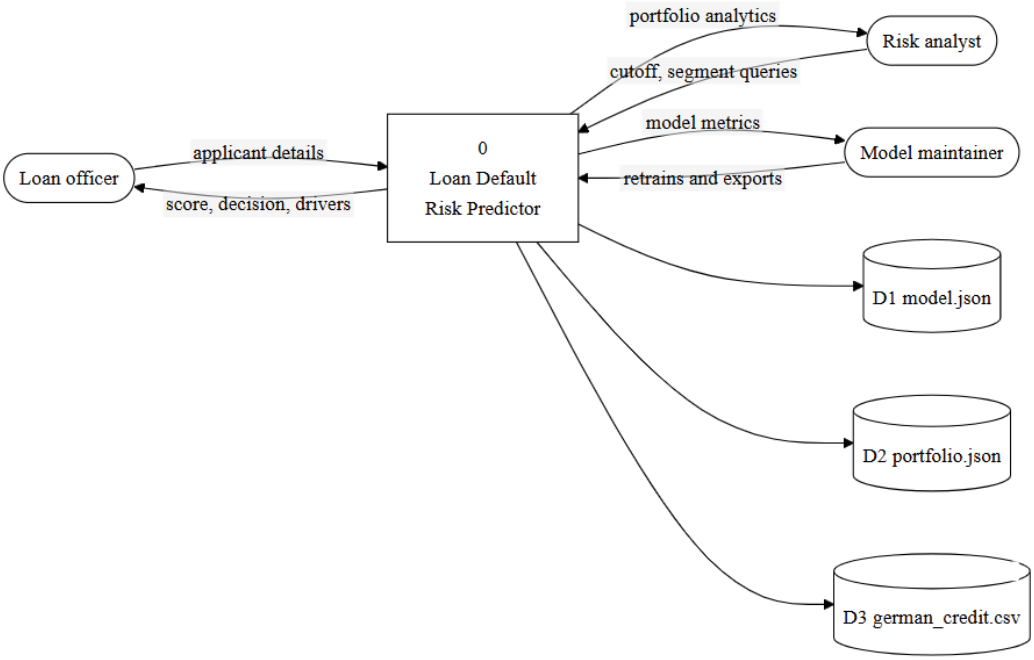


Figure 3.4 Context Level DFD (Level 0)

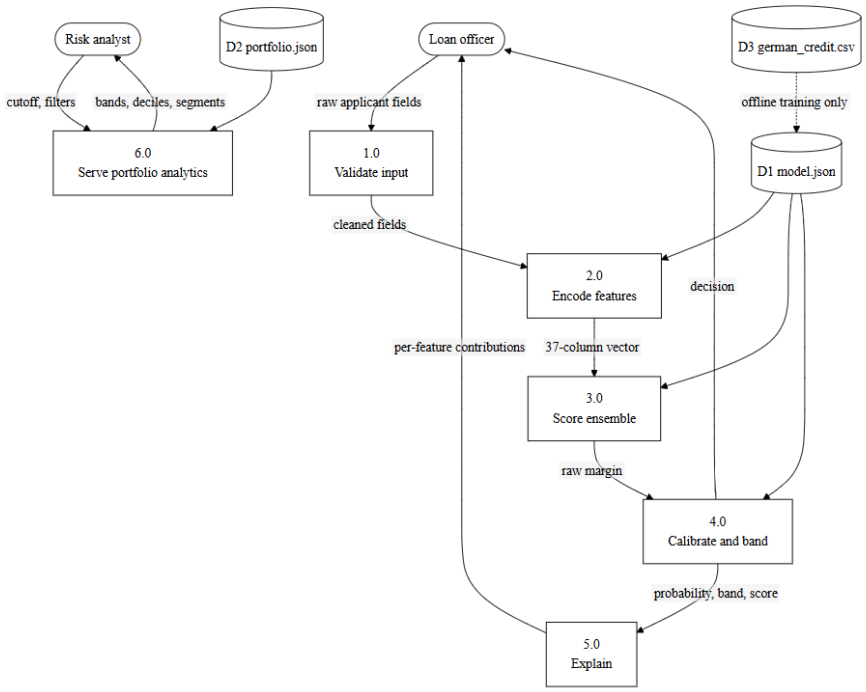


Figure 3.5 Level 1 DFD

Table 3.5 Description of Level 1 processes

No.	Process	Description
1.0	Validate input	Checks that all twelve fields are present, numeric fields are in bounds, categorical fields are known, and the instalment-to-income rule holds; rejects with a field-level 400 otherwise.
2.0	Encode features	Builds the 37-column vector from the validated fields, in the exact numeric-then-categorical order recorded in D1.
3.0	Score ensemble	Walks all 250 trees of the gradient boosting ensemble, accumulating a raw log-odds margin and a per-feature path contribution.
4.0	Calibrate & band	Applies Platt scaling to the raw margin to obtain a probability of default, then assigns a risk band and a decision using the stored cutoff.
5.0	Explain	Groups the per-feature contributions by source field and rescales them so they sum exactly to the probability minus the base rate.
6.0	Serve portfolio analytics	Reads D2 and returns the precomputed cutoff curve, deciles, calibration bins, feature importance and segments without any further computation.

Data stores: D1 model.json (the fitted ensemble and its metadata), D2 portfolio.json (the precomputed, out-of-fold portfolio aggregates), and D3 german_credit.csv (the training dataset, read only by the offline training pipeline).

Use Case Diagram

Three actors use the system: a loan officer, who scores individual applicants; a risk analyst, who studies the portfolio-level analytics; and a model maintainer, who retrains and redeploys the artifact. Their use cases are shown in Figure 3.6.

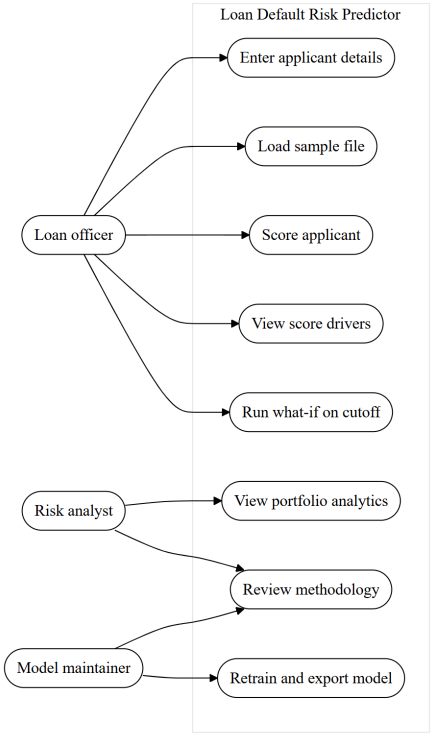


Figure 3.6 Use case diagram

Table 3.6 Use case description: Assess an applicant

Item	Description
Actor	Loan officer
Pre-condition	The officer has opened the Assess page and has the applicant's twelve field values, or chooses a sample file instead.
Main flow	1. The officer fills in the applicant form, or loads a sample file. 2. The officer submits the form. 3. The browser posts the twelve fields to `/api/predict`. 4. The server validates, scores, calibrates and explains the result. 5. The server returns the probability of default, the risk band, the score, the recommended decision and the per-factor contributions. 6. The browser renders the risk slip, the PD meter and the contribution waterfall.
Alternate flow	Validation fails: the offending field is highlighted with an inline message and focus moves to it, and nothing is submitted. Scoring service unreachable: a friendly error replaces the result panel and the form stays filled in so the officer can retry.
Post-condition	A risk slip with the probability of default, the band, the score and the decision is shown, and the officer may move the what-if cutoff slider to see how the recommendation would change.

Process Flowcharts

Figure 3.7 is the flowchart of the offline training pipeline. Figure 3.8 is the flowchart of a single scoring request. Figure 3.9 is a state diagram of the decision an applicant's probability of default is mapped onto at the deployed cutoff.

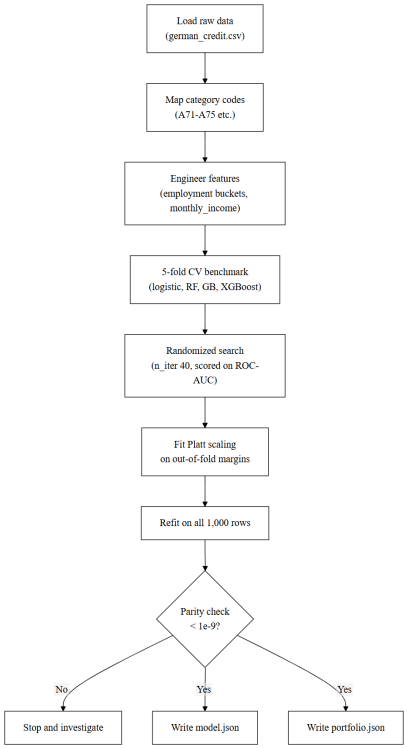


Figure 3.7 Flowchart of the training process

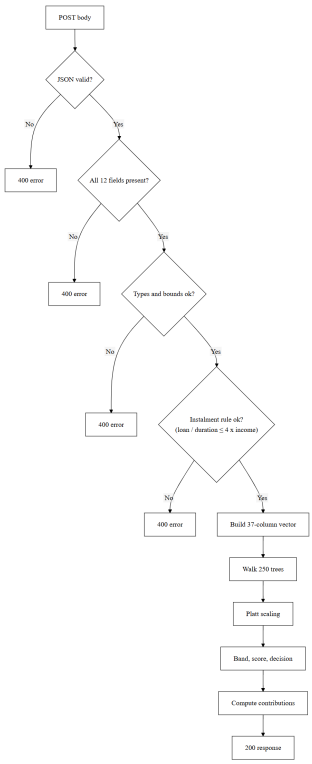


Figure 3.8 Flowchart of the scoring process

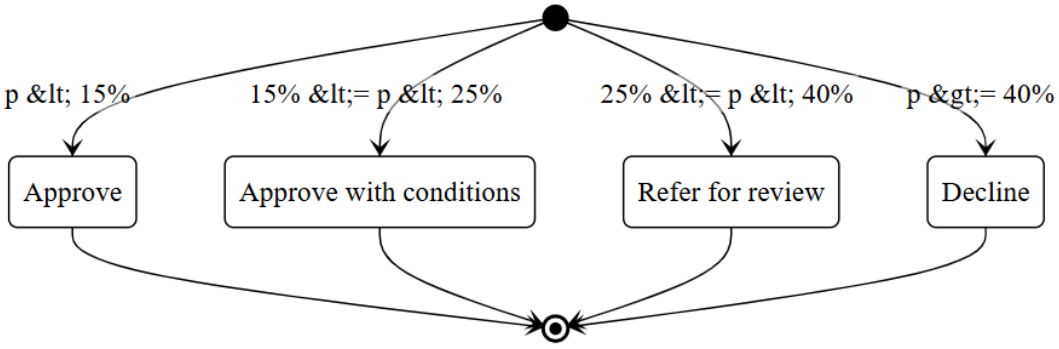


Figure 3.9 State diagram of an applicant's decision

3.3 Input Design

All data enters the system through the applicant form on the Assess page, which is divided into three sections. Required fields and bounded number inputs give the officer quick feedback in the browser, and every rule is checked again on the server, since the browser's checks can be bypassed by calling the API directly.

Table 3.7 Applicant, loan and credit position sections

Section	Field	Control	Validation
Applicant	Age	Number box (years)	Required; 18–100
Applicant	Years employed	Number box (years, 0.5 step)	Required; 0–50
Applicant	Job type	Drop-down	Required; one of four trained categories
Applicant	Housing	Drop-down	Required; one of three trained categories
Applicant	Monthly income	Number box (DM)	Required; 1–1,000,000
Loan	Amount	Number box (DM)	Required; 1–1,000,000
Loan	Duration	Number box (months)	Required; 1–120
Loan	Purpose	Drop-down	Required; one of ten trained categories
Credit position	Credit history	Drop-down	Required; one of five trained categories
Credit position	Checking account	Drop-down	Required; one of four trained categories
Credit position	Savings account	Drop-down	Required; one of five trained categories
Credit position	Existing credit lines	Number box	Required; 0–20

A cross-field rule runs after the four sections: the implied monthly instalment (amount divided by duration) may not exceed four times the monthly income, and the error is attached to the amount field when it does.

Table 3.8 Sample files and what-if controls

Field	Control	Values
Sample file	Toggle buttons	Typical file, Strong file, Stretched file; each loads a complete, realistic set of the twelve fields
What-if cutoff	Slider	5%–60%, 1-point steps, starting at the deployed cutoff (25%)

3.4 Output Design

Output is presented as a risk slip and a set of linked charts rather than a single number. The following outputs are described below, and the corresponding screenshots are attached in Annexure A-2.

- The risk slip: the probability of default, the risk band, the 300–850 score and the recommended decision (approve, approve with conditions, refer for review, decline), shown together as soon as a score comes back.
- The PD meter: the probability of default drawn on a 0–100% scale against the low/medium/high band edges.
- The decision stamp: the recommended decision rendered in the band's colour, never in the brand green, so that "approved" and "on-brand" are never confused.
- The contribution waterfall: the twelve factors' contributions to the probability, drawn on a diverging percentage-point scale so that factors pushing the score down and up are both legible.
- The what-if cutoff control: moves the approval cutoff and recomputes only the decision label, leaving the probability of default untouched.
- The risk-band split, the cutoff explorer, the decile chart and table, and the segment explorer, all on the portfolio page, each reading the precomputed aggregates in `model/portfolio.json`.
- The methodology page's benchmark table, ROC chart, calibration chart and importance bars, each explained in §3.6.
- Error messages, shown inline next to the offending field or as a form-level banner, for example "Enter a value from 18 to 100." and "The monthly instalment (amount ÷ months) can't be more than four times monthly income."

3.5 Code Design and Development

3.5.1 REST API Design

The system exposes two read endpoints, each implemented as a standard-library Python function, listed in Table 3.9.

Table 3.9 REST API endpoints

Verb	Endpoint	Access	Purpose
POST	/api/predict	Public	Score one applicant and return the probability, band, score, decision and contributions
OPTIONS	/api/predict	Public	CORS preflight; returns 204
GET	/api/portfolio	Public	Return the precomputed portfolio aggregates, cached for one hour
OPTIONS	/api/portfolio	Public	CORS preflight; returns 204

The prediction response's fields are listed in Table 3.10.

Table 3.10 Prediction response fields

Field	Type	Description
probability_of_default	Number	The calibrated probability, rounded to four decimal places.
risk_band	String	"low", "medium" or "high".
score	Integer	850 minus 550 times the probability, rounded.
decision	String	"approve", "approve_with_conditions", "refer_for_review" or "decline".
cutoff	Number	The deployed approval cutoff used for the decision.
contributions	Array of Object	One entry per source field: feature, label, value and contribution, sorted by the size of the contribution.
base_rate	Number	The dataset's observed default rate (0.30).
model_version	String	The deployed artifact's version.

3.5.2 Security Design

Security was treated as a design requirement from the start, even though the system holds no user accounts and no personal data:

- Strict input validation: every field is checked for presence, type and range before anything else runs, and an unseen category is rejected rather than coerced.
- A single error shape: every rejected request returns HTTP 400 with `{error, message}` and no stack trace; there are no 500 responses for bad input, since the handler guards the whole request body in a try/except.

- Statelessness: nothing an applicant submits is written anywhere; each request is scored and forgotten, which removes an entire class of data-breach risk.
- No secrets: the deployed functions need no API key, database password or signing secret, so there is nothing to leak.
- Open CORS by design: `Access-Control-Allow-Origin: *` is set on both endpoints. This is acceptable only because both endpoints are read-only, stateless and free of personal data; a mutating or authenticated endpoint would need a narrower origin policy.
- The protected attribute (personal status and sex) is excluded from the model, so it cannot leak into a decision even indirectly through the API's output.
- Deployment hygiene: `vercel.json`'s `excludeFiles` keeps the training dataset, the notebook and the documentation out of the deployed function bundle, so the raw data never reaches the public internet through the API's code.
- Validation is mirrored, not shared: the same twelve bounds and the same instalment rule are enforced independently in `assess-form.tsx` and in `api/predict.py`, so a request can never bypass the rule by skipping the browser.

3.5.3 Important Code Segments

Scoring loop (api/predict.py). Each tree's path contribution is attributed from a virtual root value of 0, not the tree's real root value, which is what makes the contributions exactly additive across the whole ensemble:

```
contributions: dict[int, float] = {}
raw = model["base_value"]

for tree in model["trees"]:
    node = 0
    attributed_value = 0.0
    while tree["left"][node] != -1:
        feature_idx = tree["feature"][node]
        threshold = tree["threshold"][node]
        go_left = vector[feature_idx] <= threshold
        next_node = tree["left"][node] if go_left else tree["right"][node]
        next_value = tree["value"][next_node]

        delta = (next_value - attributed_value) * model["learning_rate"]
        contributions[feature_idx] = contributions.get(feature_idx, 0.0) +
delta

        node = next_node
        attributed_value = next_value

    raw += attributed_value * model["learning_rate"]
```

```
return raw, contributions
```

Platt calibration and the decision rule (api/predict.py). The raw margin is passed through a logistic sigmoid with the fitted A and B coefficients, and the calibrated probability is compared against the cutoff at three multipliers to choose a decision:

```
def decision_for(probability: float, cutoff: float) -> str:
    if probability < cutoff * 0.6:
        return "approve"
    if probability < cutoff:
        return "approve_with_conditions"
    if probability < cutoff * 1.6:
        return "refer_for_review"
    return "decline"


def score_to_display(probability: float) -> int:
    return round(850 - probability * (850 - 300))
```

The raw, log-odds-space contributions are rescaled afterwards into probability space so they sum exactly to the probability minus the base-rate probability, and the same bounds and instalment rule are checked a second time in the browser, in `components/assess/assess-form.tsx`, so the officer sees an inline error instead of waiting on a round trip. Both excerpts, along with the training pipeline's tree export and the Platt fit, are given in Annexure A-3.

3.6 Model Design and Development

3.6.1 Data Preparation

The training pipeline, `ml/train.py`, loads the raw UCI Statlog (German Credit Data) file, 1,000 rows with 700 good and 300 bad accounts, and keeps twelve of its twenty attributes, dropping personal status and sex as a protected attribute and six further attributes to keep the applicant form short. The employment attribute's five coded buckets (A71–A75) are mapped to numeric midpoints (0, 0.5, 2.5, 5.5 and 10 years), and monthly income is engineered from the loan amount, the duration and the instalment rate, since the dataset carries no income field of its own. The twelve retained fields are then split into six numeric columns, passed through unscaled, and six categorical columns, one-hot encoded into 31 columns, for a 37-column feature vector overall.

3.6.2 Model Selection

Four candidate models were benchmarked with five-fold stratified cross-validation, using identical folds (`StratifiedKFold(5, shuffle=True, random_state=42)`) for every

model so the comparison is fair. A manual per-fold loop was used instead of scikit-learn's scoring helpers, because the XGBoost scikit-learn wrapper raised a shape error against scikit-learn 1.9's string-named scorers. The results are given in Table 3.11 and plotted in Figure 3.10.

Table 3.11 Cross-validated benchmark of candidate models

Model	ROC-AUC (± std)	PR-AUC
Logistic regression	0.7737 ± 0.0212	0.5757
Random forest (300 trees, depth 6)	0.7883 ± 0.0277	0.6329
Gradient boosting (150 trees, depth 3, lr 0.05)	0.7859 ± 0.0188	0.6243
XGBoost (200 trees, depth 3, lr 0.05, scale_pos_weight 700/300)	0.7893 ± 0.0193	0.6299
Gradient boosting, tuned	0.7965	n/a

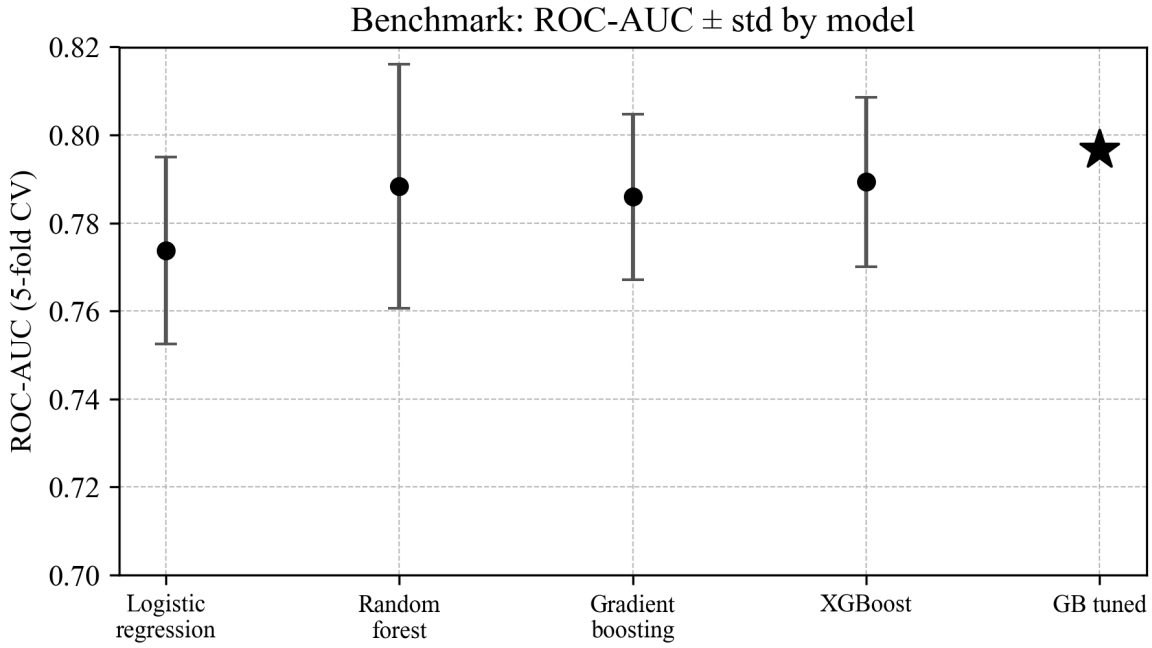


Figure 3.10 Cross-validated ROC-AUC of the candidate models

Gradient boosting was chosen over the marginally stronger untuned XGBoost and random forest results because, once tuned, it outperformed every other candidate, it needs no third-party package at inference time (the exported trees are walked by hand in pure Python), and

that absence of a runtime dependency is what lets the scoring function deploy on Vercel without scikit-learn's roughly 100 MB footprint.

3.6.3 Hyperparameter Tuning

The chosen model was tuned with `RandomizedSearchCV`, 40 iterations, scored on ROC-AUC over the same five folds. Table 3.12 gives the search space and the values selected.

Table 3.12 Search space and selected values

Hyperparameter	Search space	Selected value
n_estimators	80, 120, 150, 200, 250, 300	250
max_depth	2–5	3
learning_rate	0.01, 0.03, 0.05, 0.08, 0.1, 0.15	0.05
subsample	0.7–1.0	0.8
min_samples_leaf	5, 10, 20, 30	20

3.6.4 Probability Calibration

A gradient boosting ensemble's raw margin is a log-odds score, not a probability, and an uncalibrated score tends to be overconfident at the extremes. Platt scaling was fitted on the out-of-fold margins produced by `cross_val_predict`, giving $P = \sigma(A \cdot \text{raw} + B)$ with $A = 0.796129$ and $B = -0.113198$. At the ensemble's own starting log-odds, $\text{base_value} = \ln(0.3 / 0.7) = -0.847298$, this gives a model baseline probability of default of $\sigma(A \cdot \text{base_value} + B) \approx 0.3126$, close to the dataset's own 0.30 base rate, as Figure 3.11 confirms across the full probability range.

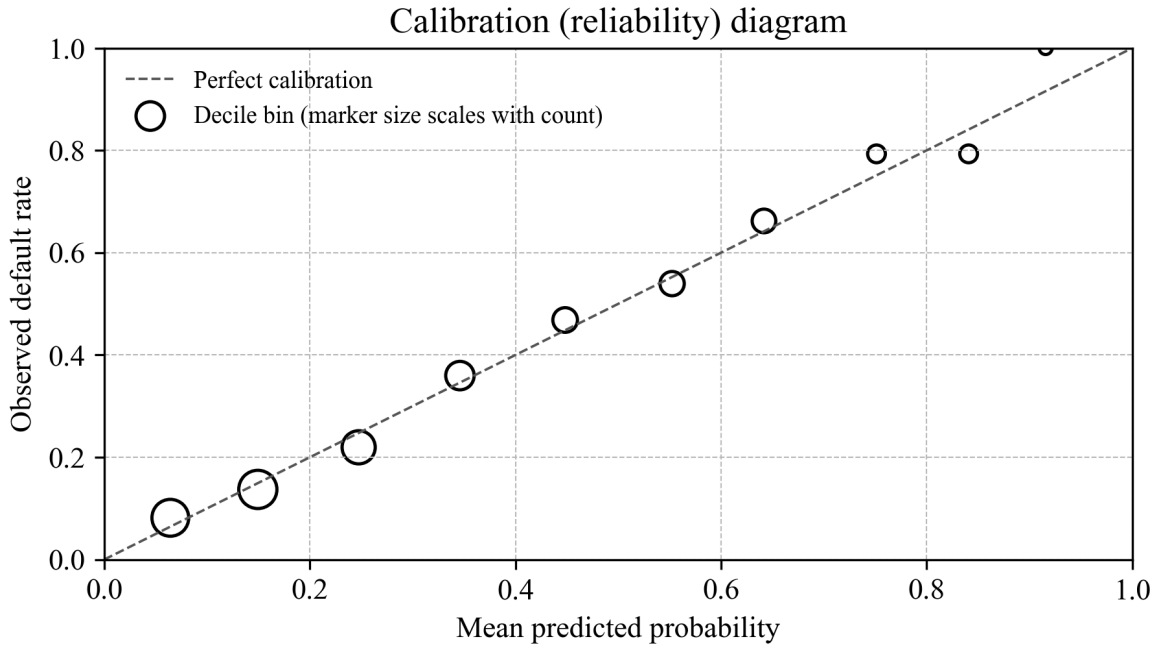


Figure 3.11 Reliability diagram of the calibrated probabilities

3.6.5 Evaluation

Every metric below is computed out-of-fold, so no applicant's score is evaluated on a model that was trained on that applicant's row. The headline metrics are given in Table 3.13.

Table 3.13 Out-of-fold evaluation metrics

Metric	Value
ROC-AUC	0.7928
PR-AUC	0.6344
KS statistic	0.4776
Brier score	0.1601 (flat-guess baseline 0.21)
Gini coefficient	0.5855
Weighted calibration gap	0.0195
Top-decile vs bottom-decile lift	10.9×
Band counts	low 327 / medium 336 / high 337

The ROC curve behind the ROC-AUC figure is plotted in Figure 3.12.

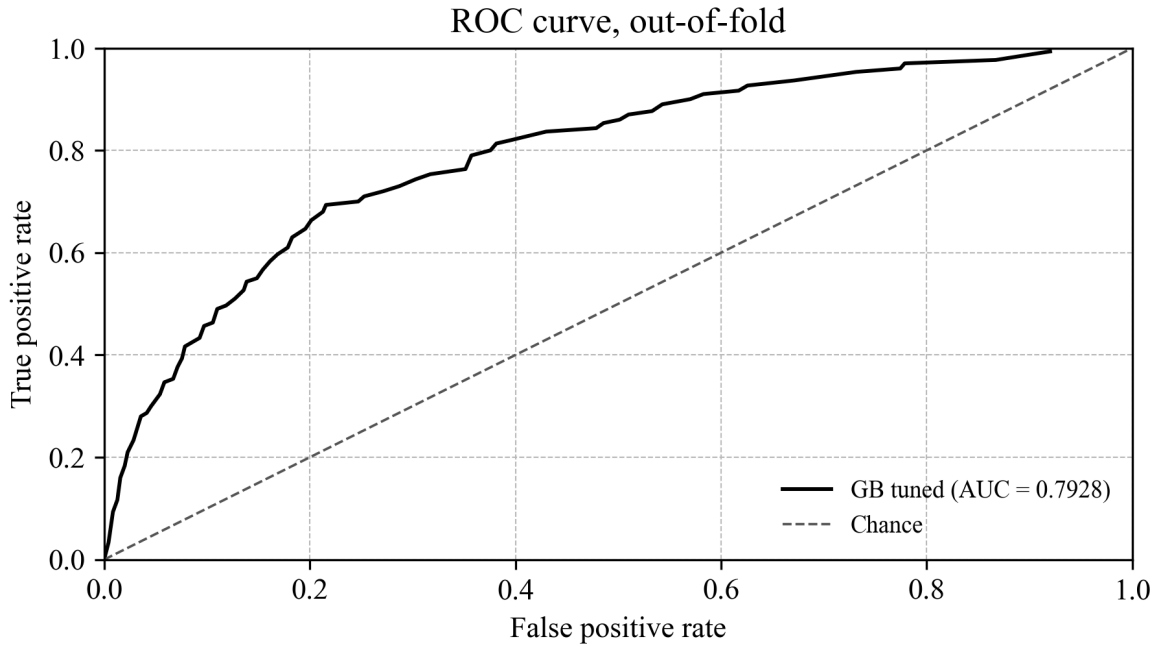


Figure 3.12 ROC curve of the calibrated model

At the deployed 0.25 cutoff, the applicants split into the confusion matrix in Table 3.14, where a predicted default is a declined application.

Table 3.14 Confusion matrix at the 0.25 cutoff

	Predicted: Decline	**Predicted: Approve**
Actual: Default	TP = 233	FN = 67
Actual: No default	FP = 250	TN = 450

This gives an accuracy of 0.683, a precision of 0.482, a recall of 0.777 and a specificity of 0.643 at a cost (five per missed default, one per wrongful decline) of 585, or 0.585 per applicant. checking_status, monthly_income and loan_amount drive most of this discrimination, as Figure 3.13's feature importance bars show, and the model's rank ordering holds up decile by decile: the top decile's observed default rate is 10.9 times the bottom decile's, as Figure 3.14 shows.

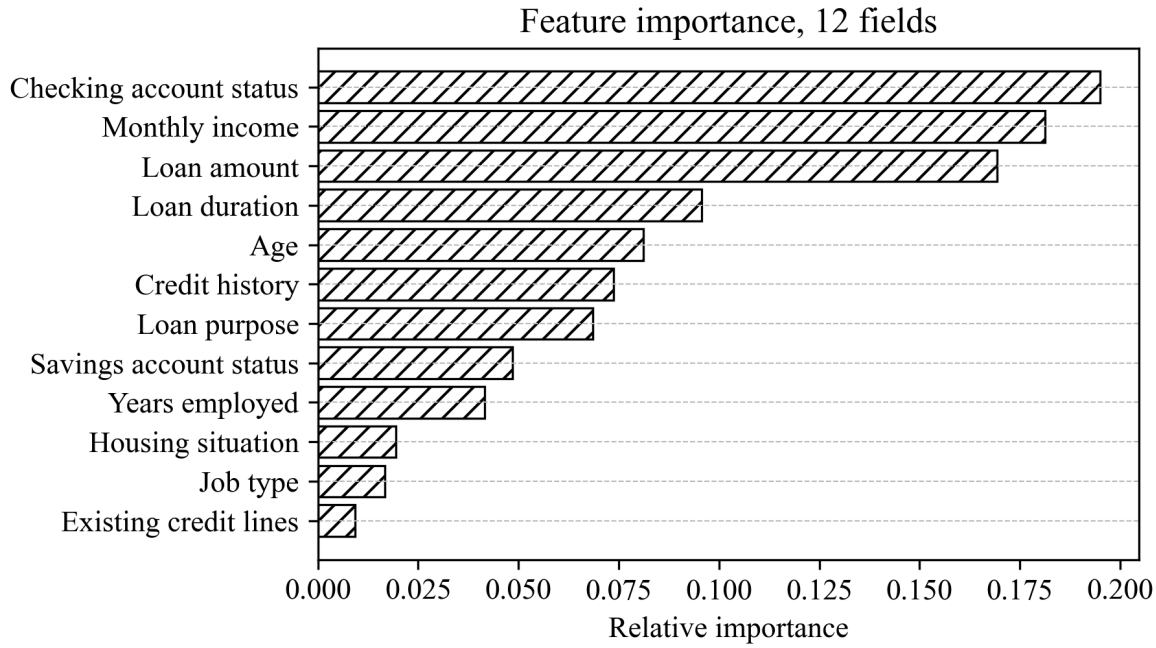


Figure 3.13 Feature importance of the twelve retained fields

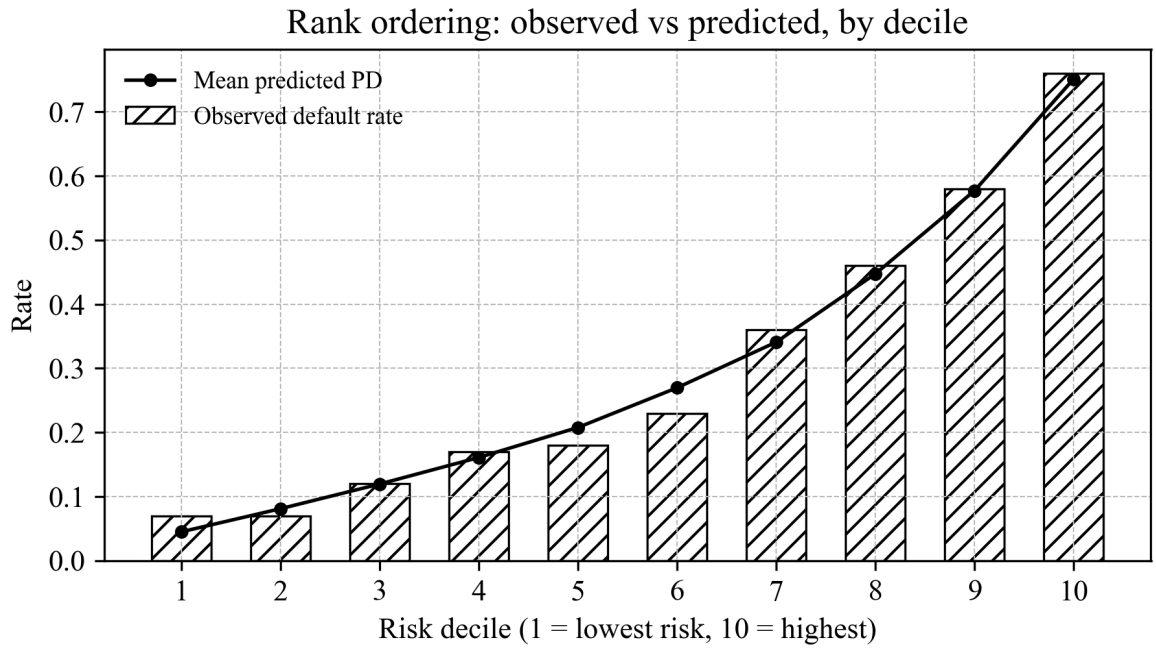


Figure 3.14 Observed versus predicted default rate by decile

3.6.6 Cutoff Selection

The approval cutoff is a business decision, not a modelling one, so it was chosen by minimising a cost function rather than by maximising a classification metric. The UCI-

documented cost matrix charges five units for an approved bad account and one unit for a wrongfully declined good account. Table 3.15 gives the approval rate, the bad rate among approved applicants, and the resulting cost at every grid point of the cutoff curve, and Figure 3.15 plots the same trade-off.

Table 3.15 Approval, bad rate and cost by cutoff

Cutoff	Approval rate	Bad rate among approved	Approved	Bads approved	Goods declined	Cost	Cost / applicant
0.05	6.3%	4.8%	63	3	640	655	0.655
0.10	20.8%	8.2%	208	17	509	594	0.594
0.15	32.7%	8.9%	327	29	402	547	0.547
0.20	43.4%	11.1%	434	48	314	554	0.554
0.25	51.7%	13.0%	517	67	250	585	0.585
0.30	59.8%	14.0%	598	84	186	606	0.606
0.35	66.3%	15.5%	663	103	140	655	0.655
0.40	71.2%	17.6%	712	125	113	738	0.738
0.45	75.1%	19.3%	751	145	94	819	0.819
0.50	79.1%	20.5%	791	162	71	881	0.881
0.55	82.6%	21.7%	826	179	53	948	0.948
0.60	86.7%	23.4%	867	203	36	1,051	1.051

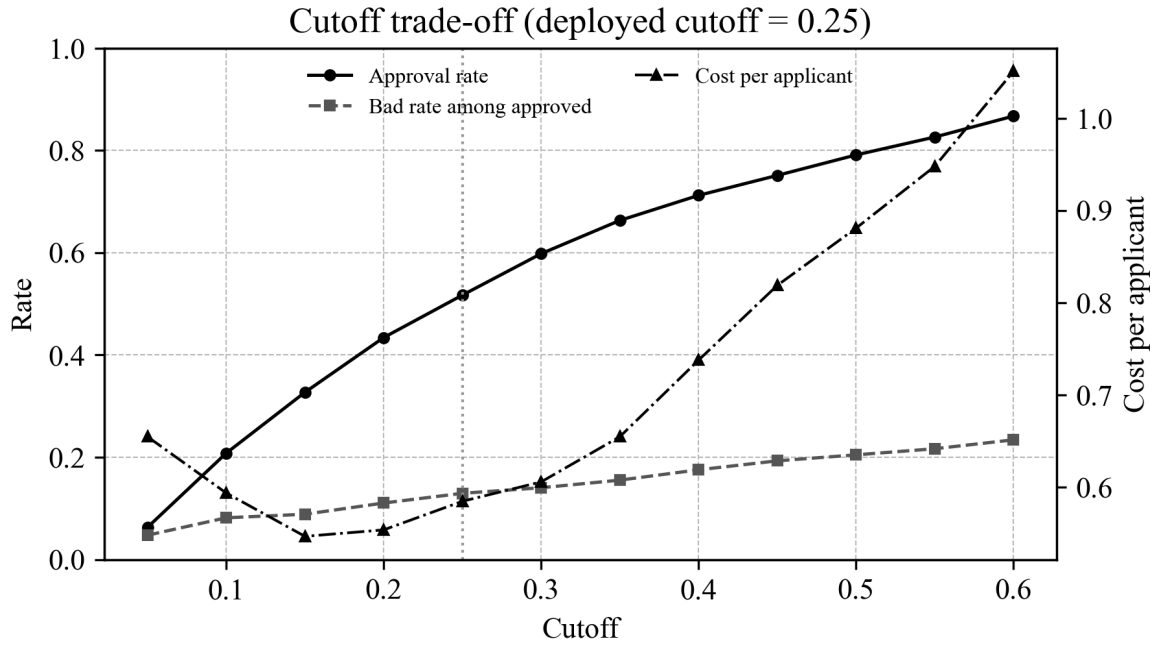


Figure 3.15 Approval rate, bad rate and cost per applicant against the cutoff

The grid's cost minimum is at a cutoff of 0.15, not 0.25. The 0.25 cutoff was kept as the deployed default even so, because it approves half the book rather than a third of it, and the grid is coarse enough (0.05 steps) that the true minimum could sit anywhere between 0.10 and 0.25; the what-if control on the Assess page exists precisely so that an officer is not locked into either choice.

3.6.7 Explaining Individual Scores

Every prediction carries its own explanation rather than relying on a separate, dataset-wide importance ranking. The method is a Saabas decomposition of the boosted ensemble: for each tree, the path an applicant's vector takes from the root to its leaf is walked, and the change in the tree's output at every split along that path is attributed to the feature that split tested, starting from a virtual value of 0 rather than the tree's actual root value. Summed across all 250 trees and scaled by the learning rate, these per-tree, per-feature deltas add up to exactly `raw_margin - base_value` in log-odds space. They are then rescaled into probability space so that they sum to `probability - base_probability`, which the notebook's additivity test confirms to within 5.77e-15.

A worked example from the documentation illustrates the method. An applicant aged 35, employed 4 years, in a skilled job, owning their home, with a monthly income of 2,500 DM,

asking for a 5,000 DM loan over 24 months for a new car, with a record of paying debts duly, a checking balance of 0–200 DM and under 100 DM in savings, and one existing credit line, scores a probability of default of 0.5718: a high-risk band, a score of 536, and a decline at the 0.25 cutoff. The largest single driver of this score is the loan amount, contributing +0.0746 to the probability, followed by the applicant's low savings balance and modest income; the contributions across all twelve fields sum to exactly the 0.2591-point difference between this probability and the model's own baseline (0.5718 − 0.3126), so the officer sees the whole explanation accounted for, not an approximation of it.

CHAPTER 4

TESTING AND IMPLEMENTATION

Learning Objectives

- Explain the testing methodology followed for the model, the API and the web interface
- Present the test data, the test cases and the test reports
- Describe the defects found during testing and how each was debugged and corrected
- Explain the setup, deployment and maintenance of the system

CHAPTER 4: TESTING AND IMPLEMENTATION

4.1 Testing Methodology

Testing of the Loan Default Risk Predictor was carried out at two levels: the machine learning pipeline that produces `model.json` and `portfolio.json`, and the Next.js and Python application that serves them. Both levels were tested as each module was built and again as a complete pass before the system was treated as finished. The following levels and techniques were used.

- Unit testing: individual functions were tested in isolation, for example `validate_and_extract()`, `score()` and `decision_for()` in `api/predict.py`, and `bandFor()` and `decisionAt()` in `lib/decision.ts`.
- Integration testing: the flow from the assessment form to `/api/predict` and back to the risk slip and the score-drivers waterfall was tested end to end, as was the flow from the portfolio and methodology pages to the build-time `portfolio.json` read.
- Black-box (functional) testing: test cases were written from the requirements, without reference to the implementation; a payload or an action was supplied and the observed result was compared with the expected one.
- White-box (structural) testing: the branches of `validate_and_extract()` and `score()` were exercised directly, so that every bound, every unknown category and the instalment cross-rule were each covered by at least one case.
- Model validation testing: the exported JSON artifact was checked against the fitted scikit-learn pipeline it was flattened from, for parity of the probability it returns, for additivity of its path contributions, and for agreement with an independent, offline TreeSHAP explanation.
- Regression testing: once a defect was fixed, the case that caught it was kept in the suite so a later change could not silently reintroduce it. Defect 1 in Table 4.7, in which path contributions were found not to sum to the model's output, is the clearest example: the additivity test (Table 4.1, T6) now runs on every notebook execution and would fail immediately if the virtual-zero convention in `score()` were ever reverted.
- Performance testing: 1,000 repeated calls to the scoring function were timed to confirm the model stays well inside interactive latency on the Vercel Python runtime.

Test Environment

Item	Detail
Operating system	Windows 11
Python	3.12.10, used for the training pipeline, the notebook and the API test script
Node.js	v24.15.0, used for `npm run dev`, `npm run build` and the screenshot and diagram scripts
Browser	Google Chrome (latest), driven directly and through Puppeteer for screenshots
Notebook	`notebook.ipynb`, re-executed with `jupyter nbconvert --execute` for an honest, reproducible run of the model-validation tests
Scripts	`report/scripts/api_tests.py` against the live `/api/predict` endpoint on `127.0.0.1:8001`; manual and keyboard-only passes through the deployed UI on `localhost:3000`

4.2 Test Data & Test Cases

Test cases were grouped by what they exercised and are given in Tables 4.1 to 4.5. Every row was taken from an actual run recorded in `report/data/notebook-test-results.json`, `report/data/api-test-results.json` or `report/data/ui-test-results.json`; none of the data or outcomes below was typed from memory. IDs run T1 to T58 continuously across the five tables, as in the sample report.

Table 4.1 Model fidelity and correctness tests

ID	Test case	Test data	Expected result	Status
T1	Artifact integrity	`model.json`: version, tree count, base value, Platt A/B, node values	version 1.0.0, 250 trees, all node values finite	Pass
T2	Parity between the stdlib scorer and scikit-learn	All 1,000 training rows scored both ways	Max |probability difference| < 1e-9	Pass (max diff 3.33e-16)
T3	Strong applicant scores a low band	Strong-file preset values	PD below cutoff, band low	Pass (PD 0.0673, band low)

ID	Test case	Test data	Expected result	Status
T4	Weak applicant scores a high band	Stretched-file preset values	Band high, decision decline	Pass (PD 0.9645, decline)
T5	Directional sanity on loan amount	Same applicant with loan amount 2,000 DM and 15,000 DM	PD rises as loan amount rises, all else equal	Pass (PD 0.3954 → 0.8503)
T6	Additivity of path contributions	100 sampled rows	base_value + Σcontributions = raw margin, within 1e-9	Pass (max diff 5.77e-15)
T7	Runtime contributions agree with offline TreeSHAP	100 sampled applicants, top-5 factors each way	Most applicants should agree on at least 3 of their top 5 factors	Pass (93/100, 93%)

Table 4.2 API validation and robustness tests

ID	Test case	Test data	Expected result	Status
T8	Valid typical applicant	12-field typical payload	200, full result shape	Pass
T9	Valid strong-file applicant	Strong-file payload	200, low PD, low band	Pass (PD 0.0325, approve)
T10	Valid stretched-file applicant	Stretched-file payload	200, high PD, decline likely	Pass (PD 0.9589, decline)
T11	Missing required field: age	Payload without `age`	400, error field age	Pass
T12	Missing required field: checking_status	Payload without `checking_status`	400, error field checking_status	Pass
T13	Missing required field: loan_amount	Payload without `loan_amount`	400, error field loan_amount	Pass
T14	Wrong type: age as a string	`age: "35"`	400, 'age' must be a number	Pass

ID	Test case	Test data	Expected result	Status
T15	Boolean passed as a number	`existing_credits: true`	400, boolean rejected as a number	Pass
T16	NaN string for a numeric field	`monthly_income: "NaN"`	400, must be a number	Pass
T17	Unknown category: housing	`housing: "mansion"`	400, unknown category	Pass
T18	Unknown category: purpose (A47 vacation)	`purpose: "vacation"`	400, unknown category	Pass
T19	Age boundary: 17	`age: 17`	400, out of bounds	Pass
T20	Age boundary: 18	`age: 18`	200, accepted	Pass
T21	Age boundary: 100	`age: 100`	200, accepted	Pass
T22	Age boundary: 101	`age: 101`	400, out of bounds	Pass
T23	Duration boundary: 0	`duration_months: 0`	400, out of bounds	Pass
T24	Duration boundary: 121	`duration_months: 121`	400, out of bounds	Pass
T25	Instalment rule exactly at the boundary	Implied instalment = 4 × income	200, accepted (the rule rejects only when strictly greater)	Pass
T26	Instalment rule just past the boundary	Implied instalment > 4 × income	400, error field loan_amount	Pass
T27	Invalid JSON body	Malformed request body	400, body must be valid JSON	Pass
T28	OPTIONS preflight	OPTIONS /api/predict	204, with CORS headers	Pass
T29	CORS header on a POST response	Any valid POST	Access-Control-Allow-Origin: *	Pass
T30	Response has every expected key	Any valid POST	probability_of_default, risk_band, score, decision, cutoff, contributions, base_rate, model_version all present	Pass
T31	PD range and score formula	Any valid POST	PD in [0, 1]; score = round(850 − 550 × PD)	Pass

ID	Test case	Test data	Expected result	Status
T32	Contributions sum to PD − base PD	Any valid POST	Σcontributions = probability − base probability, within 1e-3	Pass

Table 4.3 Model quality and performance tests

ID	Test case	Test data	Expected result	Status
T33	Quality gate on out-of-fold predictions	1,000 out-of-fold probabilities	ROC-AUC ≥ 0.75, KS ≥ 0.35	Pass (AUC 0.7928, KS 0.4776, Brier 0.1601 against a 0.2100 flat-guess baseline)
T34	Calibration within 2 percentage points	10 calibration bins	Weighted mean |predicted − observed| ≤ 0.02	Pass (0.0195)
T35	Latency over 1,000 predictions	1,000 repeated calls to `predict()` on the test machine	Sub-millisecond order of magnitude, acceptable for an interactive form	Pass (mean 1.019 ms, p95 2.805 ms, max 17.836 ms; see note below)

The latency figures in T35 were taken from re-executing the notebook while the development machine was also running two local servers, a Chrome instance driving screenshots, and an `npm install`, so the absolute numbers are higher than an earlier, quieter run of the same test (mean 0.729 ms, p95 0.907 ms, max 2.767 ms). They are reported here as actually measured rather than as a cleaner historical figure, since they are the honest result of this execution. Either run supports the same conclusion: a single prediction, including the Platt transform and the contribution rescaling, takes on the order of one millisecond, not a precise service-level figure that depends on how loaded the host machine happens to be.

Table 4.4 User interface tests

ID	Test case	Test data	Expected result	Status
T36	Load the assess page	Fresh load of `/assess`	Typical preset pre-filled, empty risk slip shown	Pass
T37	Select the Strong file preset	Click the Strong file chip	All 12 fields repopulate, chip shows pressed	Pass
T38	Submit the strong applicant	Strong-file preset, submitted	Low band, Approve, waterfall sorted by magnitude	Pass (PD 3.3%, score 832/850)
T39	Submit the stretched applicant	Stretched-file preset, submitted	High band, Decline	Pass (PD 95.9%, score 323/850)
T40	Clear a required field and submit	Age field emptied	Inline required message, field focused, no request sent	Pass
T41	Out-of-range age	Age set to 15	Range message shown, no request sent	Pass
T42	Instalment rule triggered from the form	Income 500, loan 100,000, duration 6 months	Instalment-rule message on the amount field	Pass
T43	Scoring service unreachable	Python server stopped, then a submit	Friendly inline error, no crash	Pass
T44	Move the what-if cutoff slider	Slider dragged from 25% to 49%	Decision stamp updates live, Back to 25% link appears	Pass
T45	Score-drivers waterfall	Typical result, cross-checked against the docs example	Rows sorted by size, contributions sum to PD	Pass
T46	Theme toggle persistence	Toggle to light, reload	Theme persists, no flash of the wrong theme	Pass
T47	Mobile result scroll behaviour	Code review of the submit handler in assess-workbench.tsx	Result panel scrolls into view only below the lg breakpoint	Pass (verified by code review; see note below)
T48	Portfolio cutoff explorer	Default 25% cutoff, then dragged to 40%	Four readouts update correctly	Pass (517/67/250/0.585 at 25%; 712/125/113/0.738 at 40%)
T49	Portfolio segment tabs	Purpose tab, then Housing tab	Small-sample tag shown only for n < 25	Pass

ID	Test case	Test data	Expected result	Status
T50	Methodology page table of contents	Scroll past sections; click a TOC link	Scroll-spy highlight moves; anchor link jumps to the section	Pass
T51	Custom 404 page	Navigate to an unknown route	Custom not-found page renders, no console errors	Pass
T52	Keyboard-only assessment	Tab and Enter only, through the whole form	Visible focus throughout, form submits and scores correctly	Pass
T53	Purpose dropdown has no Vacation option	Open the purpose select on /assess	Vacation absent from the options	Pass

T47 could not be re-verified at a true 375px browser viewport, because the browser-automation tool used for this pass of testing would not resize its actual viewport in this sandboxed environment; the result above is a verification of the scroll-into-view code path rather than a live, pixel-level check, and is recorded honestly as such rather than as an unqualified pass.

Table 4.5 Endpoint and integration tests

ID	Test case	Test data	Expected result	Status
T54	Handler smoke test	One valid POST through the full handler	200, all eight response keys present	Pass
T55	Validation errors return 400 with a useful message	A missing field, an unknown category and a wrong type, one case each	400 in every case, with a field name and a readable message	Pass
T56	Boundary values handled without a 500	Six boundary-value cases	Only 400 or 200, never 500	Pass (statuses 400, 200, 200, 200, 400, 400)
T57	Portfolio endpoint consistency with the calibration check	GET /api/portfolio alongside the calibration test	Band counts sum to 1,000; endpoint returns 200	Pass

ID	Test case	Test data	Expected result	Status
T58	Portfolio page reads the build-time artifact correctly	Cutoff-explorer readouts on /portfolio compared against portfolio.json	Readouts match the artifact exactly at the deployed 25% cutoff	Pass (517 approved, 67 defaults, 250 good declined, cost 0.585)

4.3 Test Reports and Debugging

4.3.1 Test Report

A total of 58 test cases were prepared and executed across the model, the API and the interface. The summary is given in Table 4.6. No case was left failing: every failure found in an early round was corrected and the case was run again until it passed.

Table 4.6 Test summary

Module	Test cases	Passed	Failed after correction
Model fidelity and correctness	7	7	0
API validation and robustness	25	25	0
Model quality and performance	3	3	0
User interface	18	18	0
Endpoint and integration	5	5	0
Total	58	58	0

4.3.2 Defects Found and Debugging

Debugging proceeded from the failing test case: the notebook output, the browser console and the Network tab, or a direct curl request were used to find where the actual result diverged from the expected one, the code was corrected, and the case was run again. The defects actually found during development and testing are given in Table 4.7.

Table 4.7 Defects found and corrections made

No.	Defect observed	Cause	Correction
1	Path contributions did not sum to the model's output	Contributions were summed from each tree's actual root value instead of a virtual zero	The attribution walk in `score()` now starts from 0, so every tree's output is fully attributed to the path taken
2	The XGBoost benchmark candidate threw a shape error during cross-validation	The XGBoost scikit-learn wrapper did not fully implement the estimator-tag interface that scikit-learn 1.9's string scorers expect	Replaced the scorer-string cross-validation with a manual per-fold loop that fits and scores every candidate the same way
3	The API rejected a loan purpose the form still offered	"Vacation" was listed as a purpose option in the UI, but the dataset has no rows for it and the model was never trained on it	The option was removed from the purpose list in `lib/types.ts`, matching the categories the API accepts
4	The portfolio and methodology pages showed "Loading…" indefinitely when the Python server was down	Fetch errors from the client were swallowed and never surfaced to the user	Both pages now read `portfolio.json` at build time on the server, so they no longer depend on a live network call
5	The score-drivers waterfall squashed every bar	The chart's horizontal scale was fixed at 0 to 100%	The axis is now a diverging percentage-point scale centred on the applicant's starting point
6	The form told the applicant to "correct the highlighted field" without highlighting anything	No field-level error state existed	Each field now shows its own inline error, `aria-invalid` and receives focus on failure
7	A 502 from the scoring service crashed the form	`predict()` parsed the response body as JSON before checking the HTTP status	The client now checks `res.ok` first and only parses the body as JSON when the response is actually JSON
8	In dark mode, the brand green and the "low risk" colour were visually identical	Both used the same hex value	Risk bands were given their own hues (petrol blue, ochre, brick), deliberately outside the brand green

No.	Defect observed	Cause	Correction
9	Clearing a number field silently became 0	Number inputs were bound directly to numeric state	Fields now keep string state while being edited and are only validated and converted on submit
10	The cutoff shown in the UI stayed at 25% even if the model's cutoff changed	The 25% figure was hard-coded in the interface text	The UI now reads `model.cutoff` directly from the exported artifact

No additional defects were found during the user-interface QA pass beyond the ten above; that pass is recorded in full in `report/data/ui-test-results.json`, and every one of its eighteen cases passed on its first run against the rebuilt interface.

4.4 Implementation

4.4.1 Local Setup Steps

1. Install Python 3.12 and Node.js 24, and clone the repository.
2. Run `py -3.12 ml/train.py` once to produce `model/model.json` and `model/portfolio.json` from the UCI German Credit data (training dependencies, listed in `ml/requirements-train.txt`, are never read by the deployed API).
3. Run `npm install` in the project root.
4. Start the Python scorer and portfolio server with `py -3.12 ml/dev_server.py` (predict on `:8001`, portfolio on `:8002`).
5. In a second terminal, start the web application with `npm run dev`, which serves `:3000` and, in development only, rewrites API calls to the two Python ports via `next.config.ts`.
6. Open `http://localhost:3000` in a browser.

4.4.2 Deployment

The system is deployed on Vercel, whose build is shown in Figure 4.1.

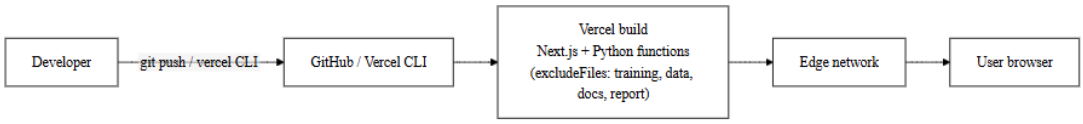


Figure 4.1 Deployment of the Loan Default Risk Predictor

- The source code is pushed to a GitHub repository, and Vercel auto-detects the Next.js application.
- `api/predict.py` and `api/portfolio.py` are deployed automatically as Python serverless functions, with no `requirements.txt`, because both files depend on nothing beyond the Python standard library.
- `vercel.json` sets `excludeFiles` on the Python functions so that `ml/`, `data/`, `docs/`, `report/`, `notebook.ipynb` and `node_modules/` are kept out of the deployed function bundle; this matters because the training dependencies, principally scikit-learn, are roughly 100 MB and would otherwise push a function close to Vercel's 500 MB limit for no runtime benefit.
- The portfolio and methodology pages read `model/portfolio.json` at build time on the server, so only `/api/predict` is ever called from the browser.
- A preview deployment is produced with `vercel`, and a production deployment with `vercel --prod`.

4.4.3 User Manual (Brief)

Table 4.8 Using the system

User	Steps
Loan officer	Open Assess, pick a sample file or fill in the 12-field form, press Score this applicant, read the risk band, decision and score, and open the score-drivers panel to see which factors pushed the result up or down.
Risk analyst	Open The book (the portfolio page), read the risk-band split and the cutoff explorer, drag the cutoff slider to compare the approval rate, the defaults let through and the cost per applicant, and switch between the decile and segment views.

User	Steps
Model maintainer	Open Methodology to review the benchmark, the calibration curve and the stated limitations; to retrain, run `ml/train.py` against updated data and redeploy the two regenerated JSON artifacts.

4.4.4 Maintenance

The model can be retrained at any time by running `ml/train.py` against an updated dataset; the script regenerates `model/model.json` and `model/portfolio.json` from scratch and re-runs its own parity and additivity checks before writing them. Every retrain should bump `model_version` so that a deployed artifact can always be identified from the API response. Because the decision thresholds in `lib/decision.ts` on the client mirror `decision_for()` in `api/predict.py` on the server, any change to the multipliers that define approve, approve-with-conditions, refer-for-review and decline must be made in both places at once. Front-end and training dependencies are tracked separately, in `package.json` and `ml/requirements-train.txt` respectively, and should be reviewed for updates on a regular cycle since the deployed function itself has no runtime dependencies to patch.

CHAPTER 5

CONCLUSION AND REFERENCES

Learning Objectives

- Summarise the work done and conclude
- State the limitations of the system
- Suggest future scope for modification
- List references and bibliography

CHAPTER 5: CONCLUSION AND REFERENCES

5.1 Summary of the Work Done

During the six-week summer training the trainee studied the problem of estimating a loan applicant's probability of default, prepared a requirement document, and built the Loan Default Risk Predictor on the UCI Statlog German Credit dataset. The work covered the full life cycle of a small machine learning application: cleaning and feature-engineering the 1,000-row dataset into twelve usable fields, benchmarking four candidate models under identical five-fold cross-validation, tuning a gradient boosting model with a randomized search over 40 parameter combinations, calibrating its output with Platt scaling fitted on out-of-fold margins, and exporting the fitted ensemble as a dependency-free JSON artifact that a stand-alone Python function could score without scikit-learn. On top of that artifact the trainee built a dependency-free scoring API with full input validation, a client explanation mechanism based on per-tree path contributions, and a Next.js front end covering an assessment workbench, a portfolio analytics page and a methodology page, styled as an editorial fintech site in the project's green. The work closed with 58 test cases across the model, the API and the interface, correction of the ten defects described in Table 4.7, and deployment of the application to Vercel.

5.2 Conclusion

The Loan Default Risk Predictor meets the objectives set out in Chapter 1. It takes the twelve fields a lender would reasonably be able to ask an applicant for and returns a calibrated probability of default, a risk band, a 300-to-850 score and a cutoff-based recommendation, each one accompanied by a per-factor explanation that sums exactly to the predicted probability rather than only approximating it. The tuned gradient boosting model reached an out-of-fold ROC-AUC of 0.7928 and a KS statistic of 0.4776, ahead of the plain logistic-regression baseline at 0.7737 and in line with the untuned random forest and XGBoost candidates evaluated under the same folds. Its Brier score of 0.1601, against a flat-guess baseline of 0.2100, and a weighted calibration gap of 0.0195 across ten probability bins, both say the same thing in different ways: the number the model returns can be read as an actual probability, not merely used to rank applicants. The top-decile applicants defaulted 10.9 times more often than the bottom-decile applicants, which is the separation a lender would need before a cutoff is worth setting at all. None of this came at the cost of transparency: every

prediction's path contributions are exactly additive by construction, and they agreed with an independent, offline TreeSHAP ranking on at least three of five top factors for 93 of 100 sampled applicants.

The project also gave the trainee practical experience that a course alone would not. Training a model honestly, with a manual per-fold cross-validation loop written because a library scorer could not be trusted across every candidate, is a different exercise from reporting a single train-test split. Exporting that model into a hand-written tree walk that reproduces scikit-learn's output to within 3.33×10^{-16} taught more about how a gradient boosting ensemble actually scores a row than fitting the model once and calling `.predict()` ever would. Writing an API with no third-party dependencies at all, so that it could run inside Vercel's 500 MB Python function limit without the roughly 100 MB that scikit-learn alone would add, forced a close reading of exactly what the trained model needs at inference time and what it does not. Building the explanation panel on an exact decomposition, rather than an approximate one, meant the trainee had to prove the additivity property in the notebook before trusting it in the interface. Throughout, excluding the protected personal-status-and-sex attribute, keeping the API stateless, and evaluating only on out-of-fold predictions were treated as non-negotiable rather than as afterthoughts.

5.3 Limitations of the System

- The dataset is the UCI Statlog German Credit data: 1,000 applicants from a single German bank in 1994. It is well suited to demonstrating a full modelling pipeline, but it is old and small, and the model's behaviour should not be assumed to generalise to current lending conditions or to a different population of borrowers.
- The personal-status-and-sex attribute was excluded as a protected attribute, and several other original attributes were dropped to keep the input form short. This keeps the model from using sex directly, but no formal fairness audit or proxy-discrimination analysis was carried out on the fields that remain.
- The deployed function has no scikit-learn, or any other heavy machine learning runtime, bundled into it. That keeps the Vercel function small, but it also means the model cannot be retrained, or its predictions monitored for drift, from the production environment itself; both require the separate, offline training pipeline.
- Evaluation is five-fold cross-validation on one static dataset split. The out-of-fold metrics in Chapter 3 are honest in the sense that no row was ever scored by a model

that had seen it during training, but there is no independent, more recent holdout set, and no live monitoring of how the model's predictions compare with actual outcomes after deployment.

- The path-contribution explanation is exact for what it decomposes, the model's own raw margin, but it is still only an approximation of the separate TreeSHAP method against which it was checked; the two agreed on most but not all of their top factors (93 of 100 sampled applicants).
- Loan and income amounts are stated in Deutsche Mark, the currency used in the original 1994 dataset, and are not converted to a current currency.
- There are no officer accounts, no login and no audit trail; any visitor to the site can run any assessment, and no record is kept of who ran what.
- Monthly income is a derived feature, computed from the loan amount, the duration and the instalment rate rather than measured directly from the applicant, so it carries whatever approximation error that derivation introduces.
- The cutoff explorer on the portfolio page only has data for the twelve cutoff values actually computed during training (0.05 to 0.60 in steps of 0.05); it cannot be queried at an arbitrary cutoff.

5.4 Future Scope for Modification

- A formal fairness audit, including a proxy-discrimination analysis of fields such as housing and employment history that could correlate with protected characteristics even though no protected attribute is used directly.
- Retraining on more recent lending data, together with reject inference to correct for the fact that a historical dataset only records outcomes for applicants who were actually approved.
- Richer explanations using full SHAP interaction values, rather than single-feature path contributions, to surface how pairs of factors jointly move a score.
- Monotonic constraints on the gradient boosting trees, so that, for example, increasing income can never be allowed to raise the predicted probability of default.
- Model monitoring in production, with drift detection on the incoming feature distributions and on the gap between predicted and observed default rates over time.

- Batch scoring through a CSV upload, so a lender could score a whole book of applicants in one pass instead of one at a time through the form.
- Officer accounts with role-based access and an audit log of every assessment run, addressing the lack of accountability noted in the limitations above.
- PDF export of an individual risk slip, so a result could be saved or attached to a file outside the application.
- Support for comparing two or more cutoffs side by side (an A/B view), rather than moving a single slider between them one at a time.

5.5 References / Bibliography

1. Hofmann, H. Statlog (German Credit Data). UCI Machine Learning Repository, 1994. https://archive.ics.uci.edu/dataset/144/statlog+german+credit+data
2. Friedman, J. H. Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5), 2001.
3. Platt, J. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. *Advances in Large Margin Classifiers*, MIT Press, 1999.
4. Lundberg, S. M. and Lee, S.-I. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems* (NeurIPS), 2017.
5. Saabas, A. Interpreting random forests. *treeinterpreter* blog post, 2014.
6. Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2011.
7. Chen, T. and Guestrin, C. XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (KDD), 2016.
8. Siddiqi, N. *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring*. Wiley, 2006.
9. Niculescu-Mizil, A. and Caruana, R. Predicting good probabilities with supervised learning. *Proceedings of the 22nd International Conference on Machine Learning* (ICML), 2005.
10. Vercel Inc. Next.js Documentation. https://nextjs.org/docs

11. Meta Platforms, Inc. React Documentation. https://react.dev
12. Vercel Inc. Python Runtime Documentation. https://vercel.com/docs/functions/runtimes/python
13. Tailwind Labs Inc. Tailwind CSS Documentation. https://tailwindcss.com/docs
14. Recharts. Recharts Documentation. https://recharts.org
15. Mozilla Foundation. MDN Web Docs. https://developer.mozilla.org

CHAPTER 6

ANNEXURES

Learning Objectives

- Attach sample inputs and screen snapshots
- Attach sample outputs and screen snapshots
- Attach important source code

CHAPTER 6: ANNEXURES

A-1 Sample Inputs

The following screens show the sample inputs given to the system, from the live, deployed interface.

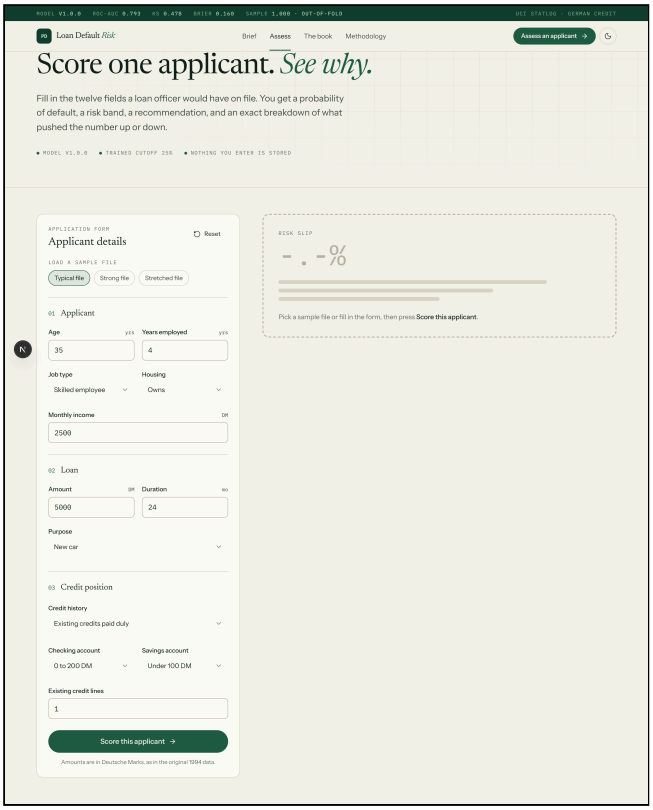


Figure A-1 Blank applicant assessment form

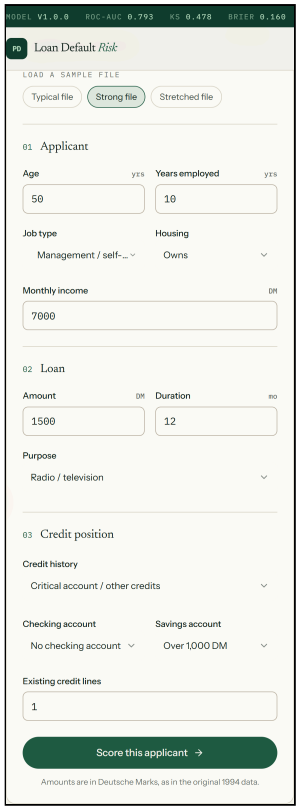


Figure A-2 A sample file loaded through a preset chip

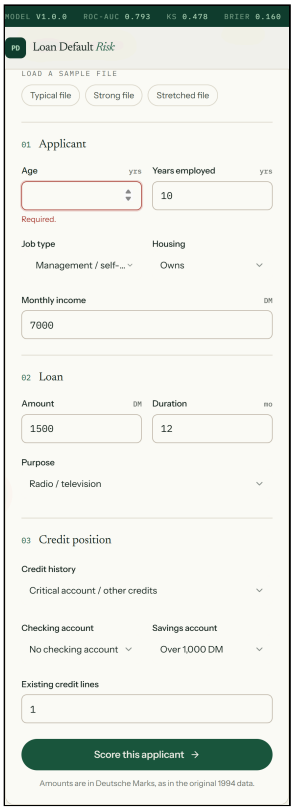


Figure A-3 A field-level validation error on submit

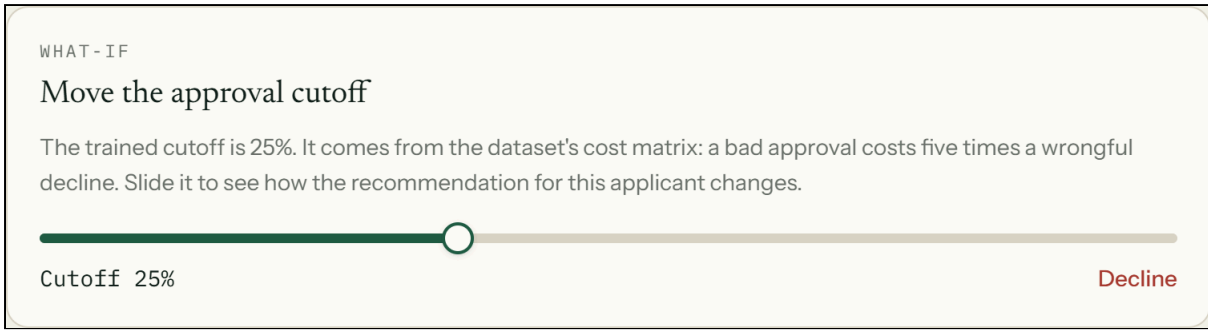


Figure A-4 The what-if cutoff control moved to 40 percent

A-2 Sample Outputs

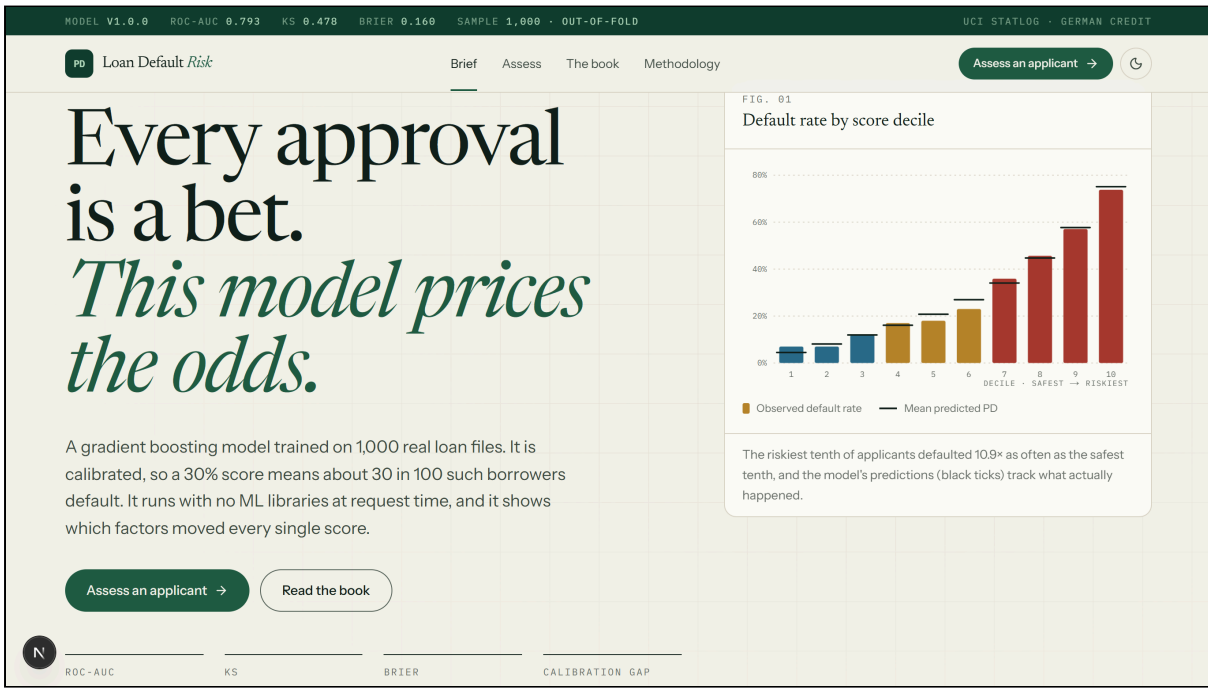


Figure A-5 The landing page

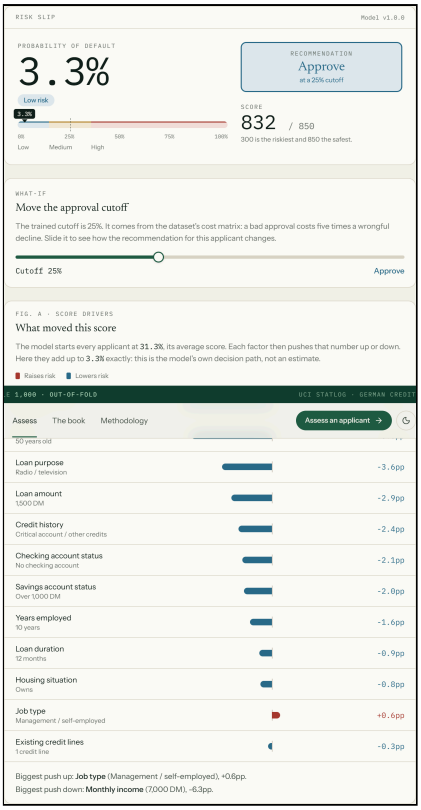


Figure A-6 A low-risk result slip

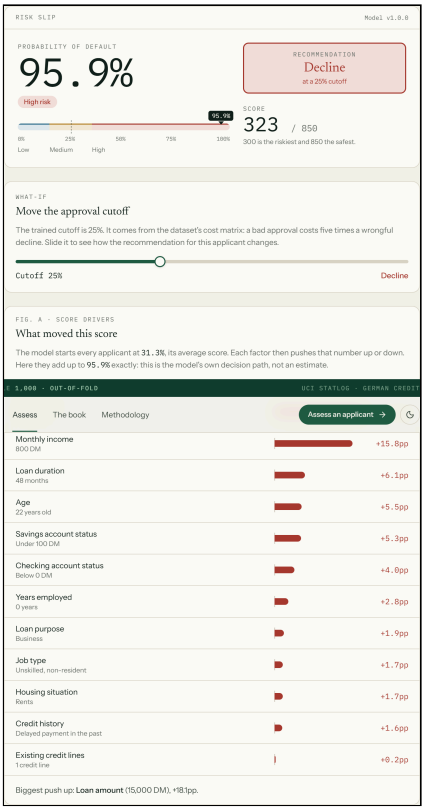


Figure A-7 A high-risk result slip

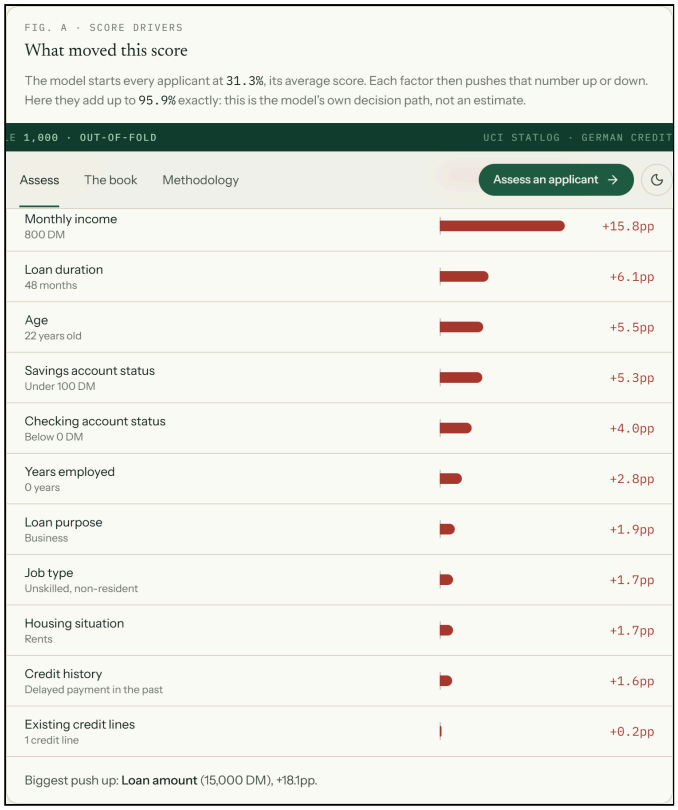


Figure A-8 The score-drivers waterfall, close up

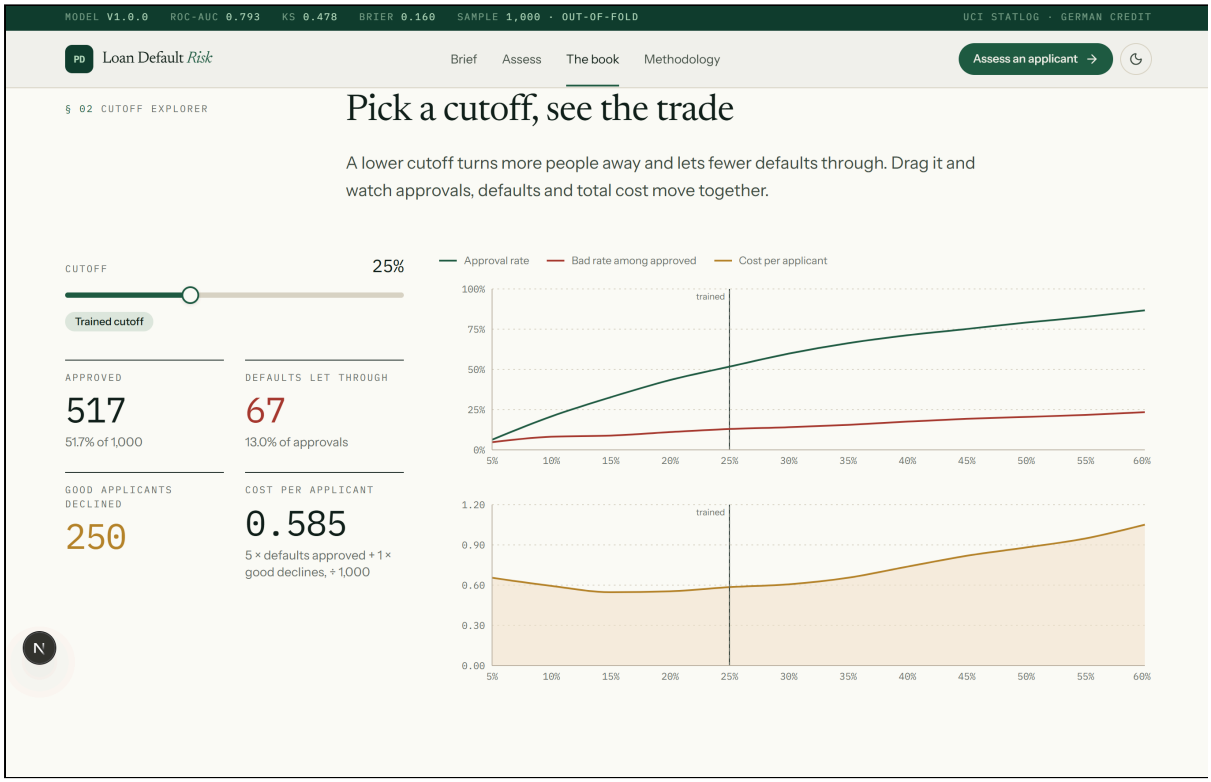


Figure A-9 The portfolio band split and cutoff explorer

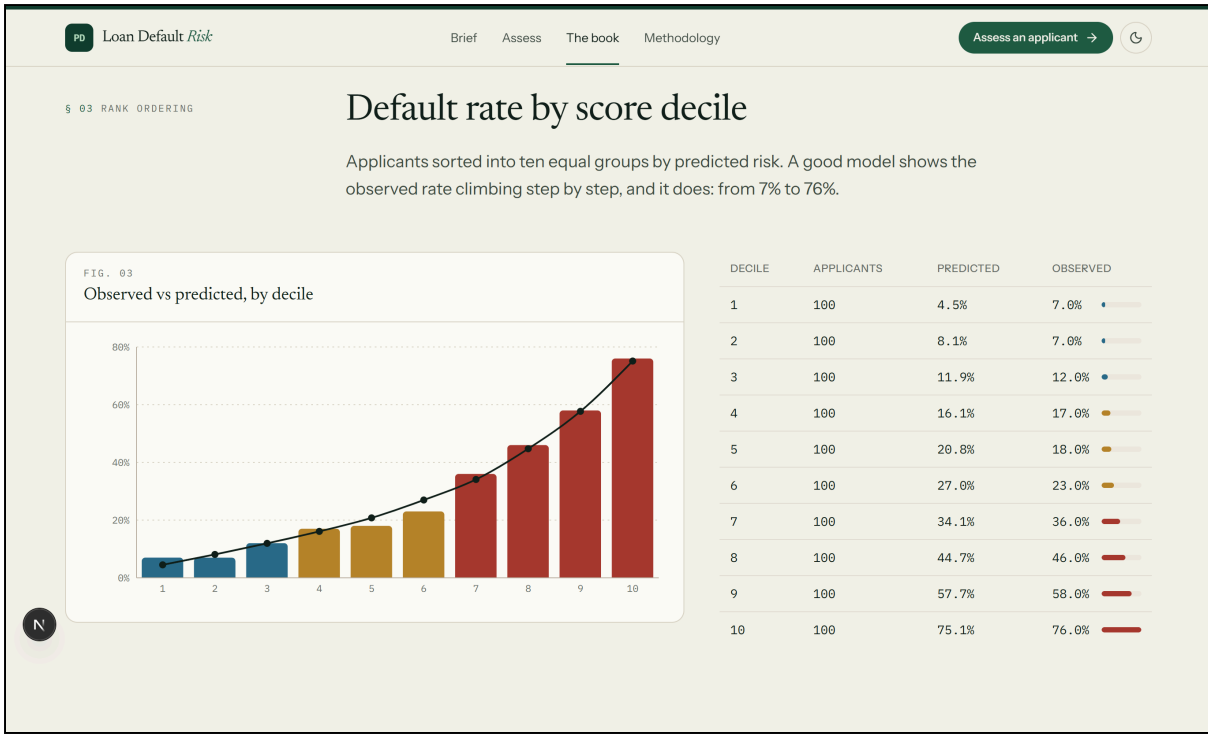


Figure A-10 The portfolio decile view

§ 02

Five candidates, identical folds

Every model was cross-validated on the same five stratified folds before any tuning, so the comparison is fair.

MODEL	ROC-AUC (MEAN)	± STD	PR-AUC	INTERVAL (0.74 TO 0.82)
Logistic regression	0.7737	± 0.0212	0.5757	
Random forest	0.7883	± 0.0277	0.6329	
Gradient boosting (default settings)	0.7859	± 0.0188	0.6243	
XGBoost	0.7893	± 0.0193	0.6299	
Gradient boosting, tuned Deployed	0.7965	n/a	n/a	

XGBoost beat untuned gradient boosting by a hair, well inside fold-to-fold noise. Tuned gradient boosting won on AUC, and it had a second advantage: its trees flatten into plain JSON that a few lines of standard-library Python can walk. The live API needs no scikit-learn at all.

Figure A-11 The model benchmark on the methodology page

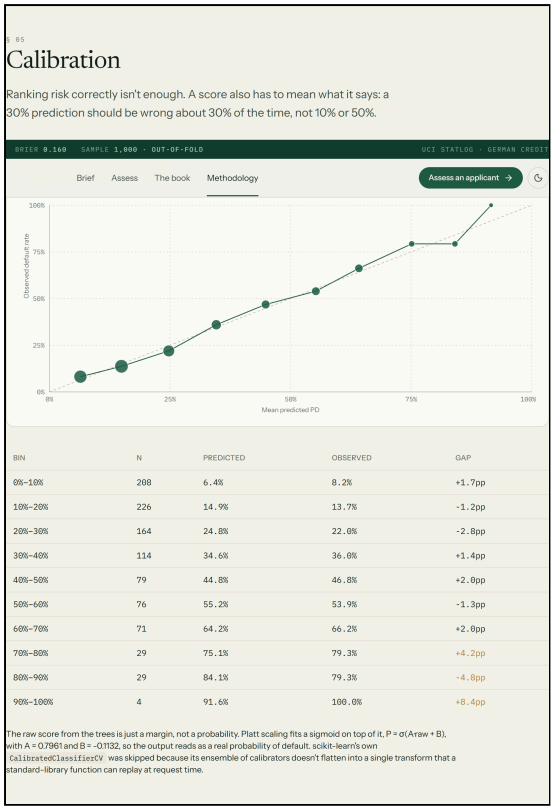


Figure A-12 The calibration curve on the methodology page

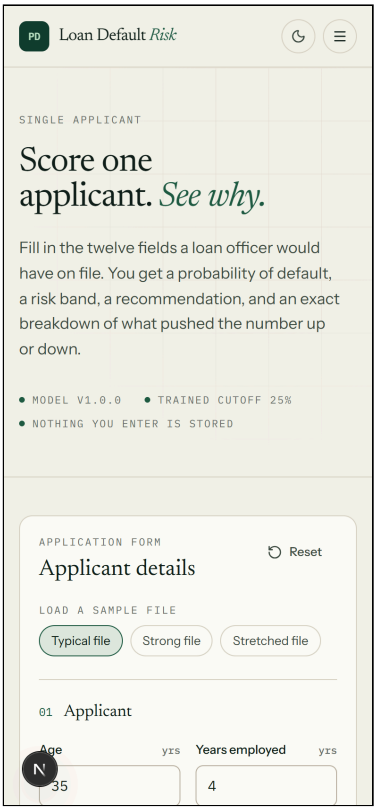


Figure A-13 The assessment workbench on a mobile screen

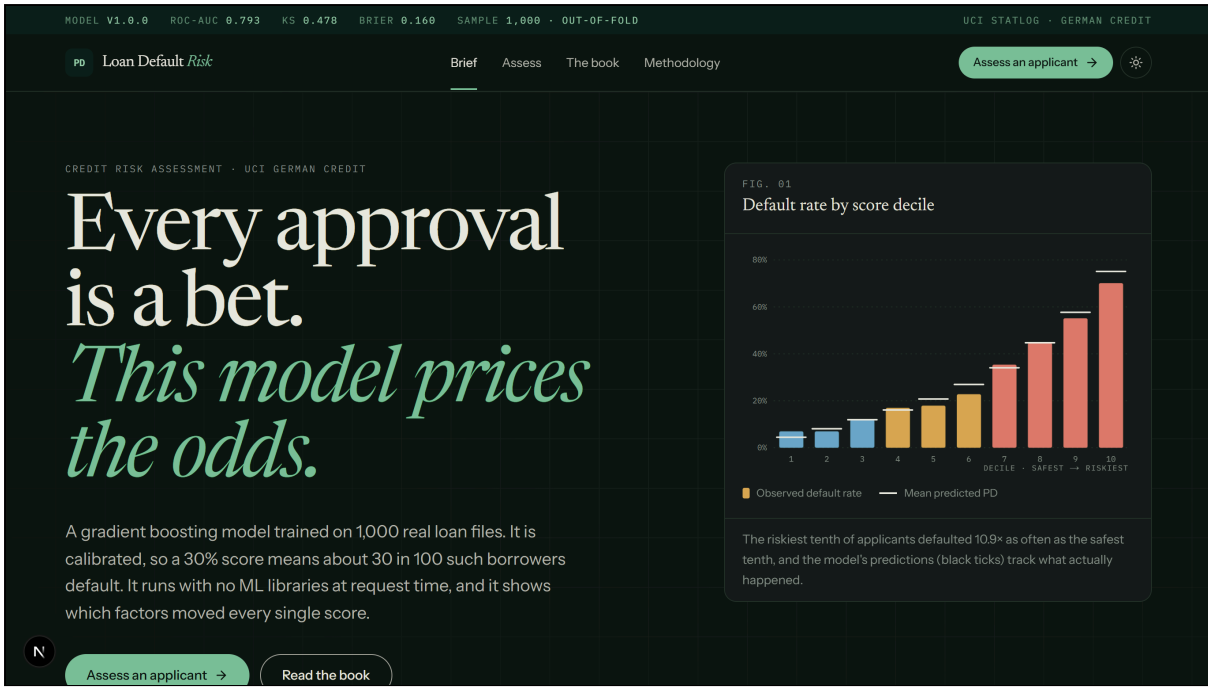


Figure A-14 The landing page in dark mode

Selected source code of the model, the scoring API and the front end is given below.

A-3 Coding

A-3.1 api/predict.py (validate_and_extract)

Every field is checked before a vector is ever built. Numeric fields are checked for type, for being finite and for their stated bounds; categorical fields are checked against the exact category list the model was trained on; and a cross-field rule rejects a loan whose implied monthly instalment is more than four times the applicant's income.

```
def validate_and_extract(payload: dict[str, Any], model: dict[str, Any]) ->
dict[str, Any]:
    if not isinstance(payload, dict):
        raise ValidationError("_body", "Request body must be a JSON object.")

    cleaned: dict[str, Any] = {}

    for field in model["numeric_features"]:
        if field not in payload:
            raise ValidationError(field, f"'{field}' is required.")
        raw = payload[field]
        if isinstance(raw, bool) or not isinstance(raw, (int, float)):
            raise ValidationError(field, f"'{field}' must be a number.")
        value = float(raw)
        if math.isnan(value) or math.isinf(value):
            raise ValidationError(field, f"'{field}' must be a finite
number.")
        lo, hi = NUMERIC_BOUNDS.get(field, (float("-inf"), float("inf")))
        if value < lo or value > hi:
            raise ValidationError(field, f"'{field}' must be between {lo} and
```

```
{hi}.")
        cleaned[field] = value

    for field in model["categorical_features"]:
        if field not in payload:
            raise ValidationError(field, f"'{field}' is required.")
        raw = payload[field]
        if not isinstance(raw, str):
            raise ValidationError(field, f"'{field}' must be a string.")
        allowed = model["categories"][field]
        if raw not in allowed:
            raise ValidationError(
                field, f"'{field}' must be one of {allowed}, got '{raw}'."
            )
        cleaned[field] = raw

    if cleaned["duration_months"] > 0:
        implied_installment = cleaned["loan_amount"] /
cleaned["duration_months"]
        if implied_installment > cleaned["monthly_income"] * 4:
            raise ValidationError(
                "loan_amount",
                "This loan amount and duration imply a monthly installment "
                "many times the applicant's income. Lower the amount, extend
"
                "the duration, or raise the income.",
            )

    return cleaned
```

A-3.2 api/predict.py (handler)

The HTTP entry point is a plain `BaseHTTPRequestHandler`, the form Vercel's Python runtime supports with no framework at all. It answers CORS preflights, reads the body, and guarantees every failure comes back as a 400 with a field and a message rather than as a 500.

```
class handler(BaseHTTPRequestHandler):
    def _send_json(self, status: int, body: dict[str, Any]) -> None:
        payload = json.dumps(body).encode("utf-8")
        self.send_response(status)
        self.send_header("Content-Type", "application/json")
        self.send_header("Access-Control-Allow-Origin", "*")
        self.send_header("Content-Length", str(len(payload)))
        self.end_headers()
        self.wfile.write(payload)

    def do_OPTIONS(self) -> None:
        self.send_response(204)
        self.send_header("Access-Control-Allow-Origin", "*")
        self.send_header("Access-Control-Allow-Methods", "POST, OPTIONS")
        self.send_header("Access-Control-Allow-Headers", "Content-Type")
        self.end_headers()

    def do_POST(self) -> None:
        try:
            length = int(self.headers.get("Content-Length", 0))
            raw_body = self.rfile.read(length) if length > 0 else b"{}"
            try:
                payload = json.loads(raw_body.decode("utf-8"))
            except (json.JSONDecodeError, UnicodeDecodeError):
                self._send_json(400, {"error": "body", "message": "Request
```

```
body must be valid JSON."})
                return

            result = predict(payload)
            self._send_json(200, result)
        except ValidationError as exc:
            self._send_json(400, {"error": exc.field, "message":
exc.message})
        except Exception as exc:  # noqa: BLE001: last-resort guard, never a
500 for bad input
            self._send_json(400, {"error": "unknown", "message": str(exc)})

    def log_message(self, format: str, *args: Any) -> None:  # noqa: A002
        pass  # keep Vercel's function logs quiet for a request this small
```

A-3.4 lib/decision.ts

The band and decision rules on the client are kept deliberately simple so they can be read at a glance, and are commented as needing to stay in step with the equivalent Python logic on the server.

```
import type { Decision, RiskBand } from "./types";

export const BAND_EDGES = { low: 0.15, medium: 0.35 } as const;

export function bandFor(pd: number): RiskBand {
  if (pd < BAND_EDGES.low) return "low";
  if (pd < BAND_EDGES.medium) return "medium";
  return "high";
}

// Mirrors decision_for() in api/predict.py; keep the multipliers in sync.
export function decisionAt(pd: number, cutoff: number): Decision {
  if (pd < cutoff * 0.6) return "approve";
  if (pd < cutoff) return "approve_with_conditions";
  if (pd < cutoff * 1.6) return "refer_for_review";
  return "decline";
}

export const DECISION_TONE: Record<Decision, "low" | "medium" | "high"> = {
  approve: "low",
  approve_with_conditions: "low",
  refer_for_review: "medium",
  decline: "high",
};
```

A-3.7 components/assess/contribution-waterfall.tsx (core)

Each row's bar is drawn out from a centre line, growing right for a risk-raising factor and left for a risk-lowering one, with its width scaled against the single largest contribution in the list so the chart stays legible regardless of how big or small an individual applicant's drivers are.

```
      <ol className="divide-y divide-hairline">
        {rows.map((r) => {
          const w = (Math.abs(r.contribution) / max) * 50;
          const pos = r.contribution >= 0;
          return (
            <li key={r.feature} className="grid grid-cols-
[minmax(0,1fr)_minmax(0,1.1fr)_4.25rem] items-center gap-3 px-5 py-2.5">
```

```
              <div className="min-w-0">
                <p className="truncate text-sm">{featureLabel(r.feature)}</p>
                <p className="truncate text-xs text-muted-foreground">
{formatFeatureValue(r.feature, r.value)}</p>
              </div>
              <div className="relative h-5" aria-hidden>
                <div className="absolute inset-y-0 left-1/2 w-px bg-hairline-
strong" />
                <div
                  className={pos ? "absolute inset-y-1 left-1/2 rounded-r-sm
bg-band-high" : "absolute inset-y-1 right-1/2 rounded-l-sm bg-band-low"}
                  style={{ width: `${Math.max(w, 0.6)}%` }}
                />
              </div>
              <p className={`num text-right text-sm ${pos ? "text-band-high"
: "text-band-low"}`}>{formatSigned(r.contribution)}</p>
            </li>
          );
        })}
      </ol>
```